

```

// File: lib\ai\hunyuan\hunyuan_image_client.dart
import 'dart:convert';
import '../tencentcloud/tencent_api_client.dart';
import '../tencentcloud/tencent_credentials.dart';
import '../tencentcloud/embedded_public_hunyuan_credentials.dart';
class HunyuanImageClient {
  static const String _host = 'aiart.tencentcloudapi.com';
  static const String _service = 'aiart';
  static const String _version = '2022-12-29';
  static const String _region = 'ap-guangzhou';
  final TencentApiClient _api;
  final TencentCredentials _credentials;
  HunyuanImageClient({
    TencentApiClient? api,
    required TencentCredentials credentials,
  }) : _api = api ?? TencentApiClient(),
      _credentials = credentials;
  String _truncateUtf8(String s, {required int maxBytes}) {
    if (maxBytes <= 0) return "";
    final raw = s.trim();
    final rawBytes = utf8.encode(raw);
    if (rawBytes.length <= maxBytes) return raw;
    final out = StringBuffer();
    int used = 0;
    for (final r in raw.runes) {
      final ch = String.fromCharCode(r);
      final b = utf8.encode(ch).length;
      if (used + b > maxBytes) break;
      out.write(ch);
      used += b;
    }
    return out.toString().trimRight();
  }
  /// 提交文生图任务
  /// 返回 JobId
  Future<String> submitTextToImageJob({
    required String prompt,
    String resolution = '1024:1024',
    int revise = 1,
    int styles =
      201, // 201: 日系动漫, 202: 3D, 203: 水墨, 204: 油画, ... (这里先给个默认值, 后续
    可扩展)
  }) async {
    final resp = await _api.postJson(

```

```

        host: _host,
        service: _service,
        action: 'SubmitTextToImageJob',
        version: _version,
        region: _region,
        secretId: _credentials.secretId,
        secretKey: _credentials.secretKey,
        useScfProxy: !usingPersonalTencentKeys(),
        payload: {
            'Prompt': _truncateUtf8(prompt, maxBytes: 1024),
            'Resolution': resolution,
            'Revise': revise,
            // 'Styles': [styles.toString()], // 可选，暂时不传，由 prompt 控制
        },
    );
    final jobId = resp['JobId'];
    if (jobId == null || jobId.toString().isEmpty) {
        throw Exception('Failed to get JobId from response: $resp');
    }
    return jobId.toString();
}

/// 查询任务状态
/// 返回包含 JobStatusCode, ResultImage 等字段的 Map
/// JobStatusCode:
/// 1: 初始化
/// 2: 处理中
/// 3: 生成中
/// 4: 失败
/// 5: 成功
Future<Map<String, dynamic>> queryTextToImageJob(String jobId) async {
    final resp = await _api.postJson(
        host: _host,
        service: _service,
        action: 'QueryTextToImageJob',
        version: _version,
        region: _region,
        secretId: _credentials.secretId,
        secretKey: _credentials.secretKey,
        useScfProxy: !usingPersonalTencentKeys(),
        payload: {
            'JobId': jobId,
        },
    );
    return resp;
}

```

```

    }
}

// File: lib\ai\hunyuan\hunyuan_text_client.dart
import '../tencentcloud/tencent_api_client.dart';
import '../tencentcloud/tencent_credentials.dart';
import '../tencentcloud/embedded_public_hunyuan_credentials.dart';
/// 流式输出返回的数据结构
class ChatStreamChunk {
  final String content;
  final String? reasoningContent;
  final bool isReasoning;
  final bool isComplete;
  ChatStreamChunk({
    required this.content,
    this.reasoningContent,
    this.isReasoning = false,
    this.isComplete = false,
  });
}

class HunyuanTextClient {
  static const String _host = 'hunyuan.tencentcloudapi.com';
  static const String _service = 'hunyuan';
  static const String _version = '2023-09-01';
  static const String _region = 'ap-guangzhou';
  final TencentApiClient _api;
  final TencentCredentials _credentials;
  HunyuanTextClient({
    TencentApiClient? api,
    required TencentCredentials credentials,
  }) : _api = api ?? TencentApiClient(),
      _credentials = credentials;
  @Deprecated('Use chatStream for better UX')
  Future<String> chatOnce({
    required String userText,
    String model = 'hunyuan-a13b',
  }) async {
    final resp = await _api.postJson(
      host: _host,
      service: _service,
      action: 'ChatCompletions',
      version: _version,
      region: _region,
      secretId: _credentials.secretId,
    );
  }
}

```

```

        secretKey: _credentials.secretKey,
        useScfProxy: !usingPersonalTencentKeys(),
        payload: {
            'Model': model,
            'Stream': false,
            'Messages': [
                {'Role': 'user', 'Content': userText},
            ],
            'EnableEnhancement': true,
            'ForceSearchEnhancement': true,
        },
    );
    final choices = resp['Choices'];
    if (choices is List && choices.isNotEmpty) {
        final first = choices.first;
        if (first is Map) {
            final msg = first['Message'];
            if (msg is Map) {
                return msg['Content']?.toString() ?? "";
            }
        }
    }
    return "";
}

Stream<ChatStreamChunk> chatStream({
    required String userText,
    String model = 'hunyuan-a13b',
    List<Map<String, String>>? messages,
}) async* {
    final stream = _api.postStream(
        host: _host,
        service: _service,
        action: 'ChatCompletions',
        version: _version,
        region: _region,
        secretId: _credentials.secretId,
        secretKey: _credentials.secretKey,
        useScfProxy: !usingPersonalTencentKeys(),
        timeout: null,
        payload: {
            'Model': model,
            'Stream': true,
            'Messages': messages ??
                [

```

```

        {'Role': 'user', 'Content': userText},
      ],
      'EnableEnhancement': true,
      'ForceSearchEnhancement': true,
    },
  );
  await for (final chunk in stream) {
    yield ChatStreamChunk(
      content: chunk.content,
      reasoningContent: chunk.reasoningContent,
      isReasoning: chunk.isReasoning,
      isComplete: chunk.isComplete,
    );
  }
}
}

```

```

// File: lib\ai\hunyuan\hunyuan_translation_engine.dart
import '../tencentcloud/tencent_api_client.dart';
import '../tencentcloud/tencent_credentials.dart';
import '../tencentcloud/tencent_cloud_exception.dart';
import '../tencentcloud/embedded_public_hunyuan_credentials.dart';
import '../translation/engines/translation_engine.dart';
class HunyuanTranslationEngine implements TranslationEngine {
  static const String _host = 'hunyuan.tencentcloudapi.com';
  static const String _service = 'hunyuan';
  static const String _version = '2023-09-01';
  static const String _region = 'ap-guangzhou';
  final TencentApiClient _api;
  final TencentCredentials _credentials;
  final String model;
  HunyuanTranslationEngine({
    TencentApiClient? api,
    required TencentCredentials credentials,
    this.model = 'hunyuan-translation',
  }) : _api = api ?? TencentApiClient(),
       _credentials = credentials;
  @override
  String get id => 'hunyuan_translation';
  @override
  Future<String> translate({
    required String text,
    required String sourceLang,
    required String targetLang,

```

```

        required List<String> contextSources,
    }) async {
        final resp = await _api.postJson(
            host: _host,
            service: _service,
            action: 'ChatTranslations',
            version: _version,
            region: _region,
            secretId: _credentials.secretId,
            secretKey: _credentials.secretKey,
            useScfProxy: !usingPersonalTencentKeys(),
            payload: {
                'Model': model,
                'Stream': false,
                'Text': text,
                if (sourceLang.trim().isEmpty) 'Source': _normalizeLang(sourceLang),
                if (targetLang.trim().isEmpty) 'Target': _normalizeLang(targetLang),
            },
            maxRetries: 0,
        );
        return _extractText(resp);
    }

    @override
    Future<List<String>> translateBatch({
        required List<String> texts,
        required String sourceLang,
        required String targetLang,
    }) async {
        final out = <String>[];
        for (final t in texts) {
            out.add(await translate(
                text: t,
                sourceLang: sourceLang,
                targetLang: targetLang,
                contextSources: const [],
            ));
        }
        return out;
    }

    String _normalizeLang(String lang) {
        final v = lang.trim();
        if (v == 'zh') return 'zh';
        if (v == 'zh-TR') return 'zh-TR';
        if (v == 'zh-Hans') return 'zh';
    }

```

```

        if (v == 'zh-Hant') return 'zh-TR';
        if (v == 'zh-TW') return 'zh-TR';
        return v;
    }
    String _extractText(Map<String, dynamic> resp) {
        final choices = resp['Choices'];
        if (choices is List && choices.isNotEmpty) {
            final first = choices.first;
            if (first is Map) {
                final msg = first['Message'];
                if (msg is Map) {
                    final c = msg['Content']?.toString();
                    if (c != null) return c;
                }
                final delta = first['Delta'];
                if (delta is Map) {
                    final c = delta['Content']?.toString();
                    if (c != null) return c;
                }
                final content = first['Content']?.toString();
                if (content != null) return content;
            }
        }
        final err = resp['ErrorMsg'];
        if (err is Map) {
            throw TencentCloudException(
                code: err['Code']?.toString() ?? 'TencentCloudError',
                message: err['Message']?.toString() ?? 'Unknown error',
            );
        }
        return "";
    }
}

```

```

// File: lib\ai\illustration\illustration_service.dart
import 'dart:async';
import 'dart:convert';
import 'dart:io';
import 'package:flutter/foundation.dart';
import 'package:http/http.dart' as http;
import 'package:uuid/uuid.dart';
import 'package:path/path.dart' as path;
import '../hunyuan/hunyuan_image_client.dart';
import '../hunyuan/hunyuan_text_client.dart';

```

```

import '../tencentcloud/tencent_credentials.dart';
import 'scene_card.dart';
class IllustrationService {
  final HunyuanImageClient _imageClient;
  final HunyuanTextClient _textClient;
  final String _baseStoragePath;
  // 轮询间隔
  static const Duration _pollInterval = Duration(seconds: 3);
  // 最大轮询次数 (约 2 分钟)
  static const int _maxPollCount = 40;
  IllustrationService({
    required TencentCredentials credentials,
    required String baseStoragePath,
  }) : _imageClient = HunyuanImageClient(credentials: credentials),
      _textClient = HunyuanTextClient(credentials: credentials),
      _baseStoragePath = baseStoragePath;
  Future<List<SceneCard>> analyzeScenesFromParagraphs({
    required List<String> paragraphs,
    required String chapterTitle,
    required int maxScenes,
    String? debugName,
    Future<String> Function(String prompt)? run,
    void Function(List<SceneCard> cards)? onProgress,
  }) async {
    final cap = maxScenes.clamp(0, 20);
    if (cap <= 0 || paragraphs.isEmpty) return const <SceneCard>[];
    final runFn = run ?? _runOnlineTextModel;
    final n = paragraphs.length;
    const int minChunkParagraphs = 2;
    final int partitionCount = (() {
      final byMinChunk = (n / minChunkParagraphs).ceil();
      return byMinChunk.clamp(1, cap);
   })();
    if (partitionCount <= 0) return const <SceneCard>[];
    final int chunkSize = (n / partitionCount).ceil().clamp(1, n);
    final List<SceneCard> allCards = [];
    final Set<int> seenIndices = {};
    for (int i = 0; i < partitionCount; i++) {
      final int start = i * chunkSize;
      final int end = (start + chunkSize).clamp(0, n);
      if (start >= end) continue;
      // Extract paragraphs for this chunk
      final subParagraphs = paragraphs.sublist(start, end);
      // Execute sequentially to avoid native crash in local LLM

```



```

final chunkCards = await _analyzeSingleChunk(
    run: runFn,
    allParagraphs: paragraphs,
    chunkStartIndex: start,
    chunkParagraphs: subParagraphs,
    chapterTitle: chapterTitle,
    debugName: '$debugName-chunk-$i',
);
var changed = false;
for (final card in chunkCards) {
    final idx = card.endParagraphIndex;
    if (idx == null) continue;
    bool isDuplicate = false;
    for (final seen in seenIndices) {
        if ((idx - seen).abs() <= 2) {
            isDuplicate = true;
            break;
        }
    }
    if (isDuplicate) continue;
    allCards.add(card);
    seenIndices.add(idx);
    changed = true;
    if (allCards.length >= cap) break;
}
if (changed) {
    allCards.sort(
        (a, b) =>
            (a.endParagraphIndex ?? 0).compareTo(b.endParagraphIndex ?? 0),
    );
    onProgress?.call(List<SceneCard>.unmodifiable(allCards));
}
if (allCards.length >= cap) break;
}
return allCards;
}
Future<List<SceneCard>> _analyzeSingleChunk({
    required Future<String> Function(String) run,
    required List<String> allParagraphs,
    required int chunkStartIndex,
    required List<String> chunkParagraphs,
    required String chapterTitle,
    required String? debugName,
}) async {

```

```

final prompt = _buildChunkPrompt(
    chunkStartIndex: chunkStartIndex,
    chunkParagraphs: chunkParagraphs,
    chapterTitle: chapterTitle,
);
if (kDebugMode) {
    debugPrint(
        '[ILLU] chunk start=$chunkStartIndex len=${chunkParagraphs.length}');
    // Log full input prompt for debugging
    debugPrint('[ILLU] PROMPT_INPUT:\n$prompt');
}
String? response;
try {
    response = await run(prompt);
    if (kDebugMode) {
        // Log full output response for debugging
        debugPrint('[ILLU] PROMPT_OUTPUT:\n$response');
    }
} catch (e) {
    debugPrint('[ILLU] chunk error: $e');
    return [];
}
// Parse result
final parsed = _parseAndValidateSceneCards(
    response,
    chapterTitle: chapterTitle,
    paragraphs: allParagraphs, // Use full list for validation safety
);
if (kDebugMode) {
    debugPrint(
        '[ILLU] parsed debugName=$debugName ok=${parsed.ok} cards=${parsed.cards.length}
hint=${parsed.errorHint}',
    );
    for (final c in parsed.cards) {
        debugPrint(
            '[ILLU] card idx=${c.endParagraphIndex} title=${c.title}
promptLen=${c.action.length}',
        );
    }
}
// Filter out invalid indices just in case (e.g. out of chunk range, though we validate against
full doc)
return parsed.cards;
}

```

```

String _buildChunkPrompt({
  required int chunkStartIndex,
  required List<String> chunkParagraphs,
  required String chapterTitle,
}) {
  final buffer = StringBuffer();
  buffer.writeln('你是一个专业的插画分镜师。');
  buffer.writeln('章节标题: $chapterTitle');
  buffer.writeln('任务: 阅读下方小说片段, 判断其中是否包含“强画面感、适合绘制插画”
的关键场景。');
  buffer.writeln('硬性要求: ');
  buffer.writeln('1) 只允许基于原文信息提炼画面; 禁止新增原文未出现的人物、地点、
道具、服饰细节。');
  buffer.writeln('2) 如果片段平淡无奇、缺乏可视化动作/环境, 只是对话或心理描写, 请
直接输出 null。');
  buffer.writeln('3) 若存在画面: 只提取 1 个最关键场景。');
  buffer.writeln('输出格式: 仅输出一个 JSON 对象 (或 null), 不要输出数组, 不要包含
解释文字。');
  buffer.writeln('JSON 字段: ');
  buffer.writeln('  - index (int): 场景对应的段落全局索引 (必须严格对应下方每行开头的
数字)。');
  buffer.writeln('  - title (string): 场景标题 (简短)。');
  buffer.writeln('  - prompt (string): 中文生图提示词 (更细更具体), 需包含: 主体、动作、环境、
时间/天气、镜头景别、光影、氛围、构图要点; 长度 <= 280 字。');
  buffer.writeln();
  buffer.writeln('小说片段 (行首数字为全局索引): ');
  for (int i = 0; i < chunkParagraphs.length; i++) {
    final globalIndex = chunkStartIndex + i;
    String p = chunkParagraphs[i];
    String normalized = _normalizeForPrompt(p, maxLen: 260);
    buffer.writeln('$globalIndex: $normalized');
  }
  return buffer.toString();
}

Future<String> _runOnlineTextModel(String prompt) async {
  final stream = _textClient.chatStream(
    userText: prompt,
    model: 'hunyuan-a13b',
  );
  final buffer = StringBuffer();
  await for (final chunk in stream) {
    if (chunk.isReasoning) continue;
    buffer.write(chunk.content);
  }
}

```

```

    }
    return buffer.toString();
}
/// 提交生图任务
Future<String> submitGeneration({
    required SceneCard card,
    required String stylePrefix,
    required bool forLocalSd,
    String resolution = '1024:1024',
}) async {
    final prompt =
        card.toPrompt(stylePrefix: stylePrefix, forLocalSd: forLocalSd);
    return await _imageClient.submitTextToImageJob(
        prompt: prompt,
        resolution: resolution,
    );
}
/// 轮询任务状态直到完成或失败
/// 返回本地文件路径
Future<String> pollJobStatus(String jobId) async {
    int count = 0;
    while (count < _maxPollCount) {
        await Future.delayed(_pollInterval);
        final status = await _imageClient.queryTextToImageJob(jobId);
        final code = status['JobStatusCode'];
        if (code == '5' || code == 5) {
            // 成功
            final urls = status['ResultImage'];
            if (urls is List && urls.isNotEmpty) {
                final url = urls.first.toString();
                if (kIsWeb) return url;
                return await _downloadImage(url, jobId);
            }
            throw Exception('Success but no image url');
        }
        if (code == '4' || code == 4) {
            // 失败
            final msg =
                status['JobErrorMsg'] ?? status['JobErrorCode'] ?? 'Unknown error';
            throw Exception('Generation failed: $msg');
        }
        count++;
    }
    throw TimeoutException('Generation timed out');
}

```

```

}
/// 下载图片到本地
Future<String> _downloadImage(String url, String jobId) async {
  if (kIsWeb) return url;
  final resp = await http.get(Uri.parse(url));
  if (resp.statusCode != 200) {
    throw Exception('Failed to download image: ${resp.statusCode}');
  }
  final dir = Directory(path.join(_baseStoragePath, 'illustrations'));
  if (!await dir.exists()) {
    await dir.create(recursive: true);
  }
  final filePath = path.join(dir.path, '$jobId.jpg');
  final file = File(filePath);
  await file.writeAsBytes(resp.bodyBytes);
  return filePath;
}

String _normalizeForPrompt(String s, {required int maxlen}) {
  final t = s.replaceAll(RegExp(r'\s+'), ' ').replaceAll('\u0000', '').trim();
  if (t.length <= maxlen) return t;
  return '${t.substring(0, maxlen)}...';
}

String _safeFileName(String s) {
  final t = s.trim().replaceAll(RegExp(r'[<: "/ \ | ? * \x00-\x1F]'), '_');
  if (t.isEmpty) return 'untitled';
  return t.length > 80 ? t.substring(0, 80) : t;
}

({bool ok, List<SceneCard> cards, String errorHint})
_parseAndValidateSceneCards(
  String raw, {
  required String chapterTitle,
  required List<String> paragraphs,
}) {
  final start = raw.indexOf('[');
  // If we can't find ']', we might be truncated.
  // Try to take everything from '[' to the end of string if ']' is missing
  final lastBracket = raw.lastIndexOf(']');
  final end = (lastBracket != -1 && lastBracket > start)
    ? lastBracket
    : raw.length - 1;
  if (start == -1 || end <= start) {
    // New logic: Allow single object or null
    final rawTrimmed = raw.trim();
    if (rawTrimmed == 'null') {

```

```

        return (ok: true, cards: const <SceneCard>[], errorHint: "");
    }
    // Compatibility: Check if it's a single object (starts with {)
    final startObj = raw.indexOf('{');
    final endObj = raw.lastIndexOf('');
    if (startObj != -1 && endObj != -1 && endObj > startObj) {
        // It is a single object
        final singleJson = raw.substring(startObj, endObj + 1);
        try {
            final decodedSingle = jsonDecode(singleJson);
            return _parseDecodedList([decodedSingle], paragraphs);
        } catch (_) {
            return (
                ok: false,
                cards: const <SceneCard>[],
                errorHint: '未找到 JSON 对象'
            );
        }
    }
    return (ok: false, cards: const <SceneCard>[], errorHint: '未找到 JSON 对象或数组');
}

final maybeTruncatedJson =
    raw.substring(start, lastBracket != -1 ? end + 1 : raw.length);
dynamic decoded;
try {
    decoded = jsonDecode(maybeTruncatedJson);
} catch (e) {
    // Try to fix common JSON errors from local models
    try {
        var fixed =
            maybeTruncatedJson.replaceAll('\n', '\\n').replaceAll('\r', "");
        // Attempt to close truncation
        if (!fixed.trim().endsWith('}')) {
            if (fixed.trim().endsWith('')) {
                fixed = '$fixed}';
            } else if (fixed.trim().endsWith('')) {
                fixed = '$fixed}]]';
            } else {
                fixed = '$fixed"']];
            }
        }
        try {
            decoded = jsonDecode(fixed);
        } catch (_) {

```

```

        // Final fallback
        final lastValidObjEnd = fixed.lastIndexOf('{,');
        if (lastValidObjEnd != -1) {
            fixed = '${fixed.substring(0, lastValidObjEnd + 1)}';
            decoded = jsonDecode(fixed);
        } else {
            throw Exception('Repair failed');
        }
    }
} catch (_) {
    if (kDebugMode) {
        debugPrint('[ILLU] JSON repair failed. Raw length: ${raw.length}');
    }
    return (ok: false, cards: const <SceneCard>[], errorHint: 'JSON 解析失败');
}

// Compatibility: If decoded is Map (single object), wrap it
if (decoded is Map) {
    decoded = [decoded];
}
if (decoded is! List) {
    return (ok: false, cards: const <SceneCard>[], errorHint: 'JSON 不是数组');
}
return _parseDecodedList(decoded, paragraphs);
}

({bool ok, List<SceneCard> cards, String errorHint}) _parseDecodedList(
    dynamic decodedList, List<String> paragraphs) {
    if (decodedList is! List)
        return (ok: false, cards: const <SceneCard>[], errorHint: '无效数据');
    final List<SceneCard> out = [];
    final List<String> errors = [];
    for (int i = 0; i < decodedList.length; i++) {
        final item = decodedList[i];
        if (item is! Map) {
            errors.add('第${i + 1}项不是对象');
            continue;
        }
        final map = item.cast<String, dynamic>();
        // Schema validation: index, title, prompt (was scene in prompt instructions, but check
        logic here)
        // The prompt asks for 'prompt' key, but let's check both for safety or just 'prompt'
        // Old code checked 'scene' because user requested 'scene' key in prompt.
        // Current prompt asks for 'prompt' key.
        String? promptVal;

```

```

if (map.containsKey('prompt')) {
    promptVal = map['prompt']?.toString();
} else if (map.containsKey('scene')) {
    promptVal = map['scene']?.toString();
}
if (!map.containsKey('index') ||
    !map.containsKey('title') ||
    promptVal == null) {
    errors.add('第${i + 1}项 缺少必要字段(index/title/prompt)');
    continue;
}
final dynamic indexRaw = map['index'];
final int? pIndex = indexRaw is int
    ? indexRaw
    : (indexRaw is String ? int.tryParse(indexRaw) : null);
if (pIndex == null || pIndex < 0 || pIndex >= paragraphs.length) {
    errors.add('第${i + 1}项 索引 $pIndex 无效');
    continue;
}
final String title = (map['title'] ?? '场景').toString();
final String scene = promptVal ?? "";
if (scene.isEmpty) {
    errors.add('第${i + 1}项 prompt 为空');
    continue;
}
out.add(SceneCard(
    id: const Uuid().v4(),
    startParagraphIndex: pIndex,
    endParagraphIndex: pIndex, // Point to single paragraph
    title: title,
    location: "",
    time: "",
    characters: "",
    action: scene, // Store prompt in action field
    mood: "",
    visualAnchors: "",
    lighting: "",
    composition: "",
    palette: "",
    createdAt: DateTime.now(),
));
}
final ok = errors.isEmpty;
final hint = errors.isEmpty ? "" : errors.take(3).join(' ');

```



```
        return (
          ok: ok,
          cards: out,
          errorHint: ok ? '' : (hint.isEmpty ? '未知错误' : hint)
        );
      }
    }
  }
}
```

// File: lib\ai\illustration\scene_card.dart

```
enum SceneCardStatus {
  draft, // 仅文字
  generating, // 正在生成图片
  completed, // 图片已生成并下载
  failed, // 生成失败
}

class SceneCard {
  final String id;
  int? startParagraphIndex;
  int? endParagraphIndex;
  final String title;
  final String location;
  final String time;
  final String characters;
  String action;
  final String mood;
  final String visualAnchors;
  final String lighting;
  final String composition;
  final String palette;
  // 生成状态相关
  SceneCardStatus status;
  String? jobId;
  String? localImagePath; // 本地图片路径
  String? errorMsg;
  DateTime? createdAt;
  SceneCard({
    required this.id,
    this.startParagraphIndex,
    this.endParagraphIndex,
    required this.title,
    required this.location,
    required this.time,
    required this.characters,
    required this.action,
```

```

        required this.mood,
        required this.visualAnchors,
        required this.lighting,
        required this.composition,
        required this.palette,
        this.status = SceneCardStatus.draft,
        this.jobId,
        this.localImagePath,
        this.errorMsg,
        this.createdAt,
    });
    // 用于序列化存储
    Map<String, dynamic> toJson() {
        return {
            'id': id,
            'startParagraphIndex': startParagraphIndex,
            'endParagraphIndex': endParagraphIndex,
            'title': title,
            'location': location,
            'time': time,
            'characters': characters,
            'action': action,
            'mood': mood,
            'visual_anchors': visualAnchors,
            'lighting': lighting,
            'composition': composition,
            'palette': palette,
            'status': status.index,
            'jobId': jobId,
            'localImagePath': localImagePath,
            'errorMsg': errorMsg,
            'createdAt': createdAt?.millisecondsSinceEpoch,
        };
    }
}

factory SceneCard.fromJson(Map<String, dynamic> json) {
    final int? start = json['startParagraphIndex'];
    final int? end = json['endParagraphIndex'];
    return SceneCard(
        id: json['id'] ?? "",
        startParagraphIndex: start,
        endParagraphIndex: end,
        title: json['title'] ?? "",
        location: json['location'] ?? "",
        time: json['time'] ?? "",
    );
}

```

```

        characters: json['characters'] ?? "",
        action: json['action'] ?? "",
        mood: json['mood'] ?? "",
        visualAnchors: json['visual_anchors'] ?? "",
        lighting: json['lighting'] ?? "",
        composition: json['composition'] ?? "",
        palette: json['palette'] ?? "",
        status: SceneCardStatus.values[json['status'] ?? 0],
        jobId: json['jobId'],
        localImagePath: json['localImagePath'],
        errorMsg: json['errorMsg'],
        createdAt: json['createdAt'] != null
            ? DateTime.fromMillisecondsSinceEpoch(json['createdAt'])
            : null,
    );
}
// 生成提示词
String toPrompt({
    required String stylePrefix,
    required bool forLocalSd,
}) {
    // New logic: prompt = style + action (which contains the full scene description)
    // The 'action' field now holds the complete prompt generated by the analysis model.
    // e.g. "A beautiful garden, sunny day..."
    final cleanAction = action.trim();
    if (cleanAction.isEmpty) {
        return stylePrefix;
    }
    // For local SD, we might want to ensure it's English?
    // But the analysis model is already instructed to output English for local SD.
    // So we just trust it.
    if (stylePrefix.isEmpty) {
        return cleanAction;
    }
    return '$stylePrefix, $cleanAction';
}
}

```

```

// File: lib\ai\licensing\license_codec.dart
import 'dart:convert';
import 'dart:math';
import 'package:cryptography/cryptography.dart';
class LicensePayload {
    final int points;

```

```

        const LicensePayload({required this.points});
    }
class LicenseException implements Exception {
    final String message;
    const LicenseException(this.message);
    @override
    String toString() => message;
}
class LicenseCodec {
    static const String _prefixPointsV3 = 'P3';
    static const List<int> _pointsByIndex = [
        50000,
        100000,
        200000,
        500000,
        1000000,
    ];
    static const int _v3NonceLength = 4;
    static const int _v3SignatureLength = 64;
    static const String publicKeyB64 = String.fromEnvironment(
        'AIRREAD_LICENSE_PUBLIC_KEY_B64',
        defaultValue: 'Z+RpD1T+mPNA3EDYVI8jJwpCTn5oEWExbhy+rbmObpc=',
    );
    static final Ed25519 _algo = Ed25519();
    static SimplePublicKey? publicKeyOverride;
    static SimplePublicKey _resolvePublicKey() {
        final override = publicKeyOverride;
        if (override != null) return override;
        final raw = publicKeyB64.trim();
        if (raw.isEmpty) {
            throw const LicenseException('未配置卡密公钥');
        }
        final bytes = base64Decode(raw);
        return SimplePublicKey(bytes, type: KeyPairType.ed25519);
    }
    static Future<LicensePayload> verifyAndParse(String code) async {
        final raw = code.trim();
        if (raw.isEmpty) throw const LicenseException('请输入卡密');
        if (raw.startsWith(_prefixPointsV3)) {
            return _verifyAndParsePointsV3(raw);
        } else {
            throw const LicenseException('卡密版本不支持');
        }
    }
}

```

```

static Future<LicensePayload> _verifyAndParsePointsV3(String raw) async {
  final body =
    raw.substring(_prefixPointsV3.length).replaceAll(RegExp(r'\s+'), '');
  if (body.isEmpty) throw const LicenseException('卡密格式错误');
  final bytes = _base64UrlDecode(body);
  const int payloadLen = 1 + _v3NonceLength;
  const int expectedLen = payloadLen + _v3SignatureLength;
  if (bytes.length != expectedLen) {
    throw const LicenseException('卡密内容无法解析');
  }
  final payload = bytes.sublist(0, payloadLen);
  final sigBytes = bytes.sublist(payloadLen);
  final index = payload[0];
  if (index < 0 || index >= _pointsByIndex.length) {
    throw const LicenseException('卡密面额不支持');
  }
  final points = _pointsByIndex[index];
  final pk = _resolvePublicKey();
  final ok = await _algo.verify(
    payload,
    signature: Signature(sigBytes, publicKey: pk),
  );
  if (!ok) throw const LicenseException('卡密校验失败');
  return LicensePayload(points: points);
}

static Future<String> generateSignedPointsFromSeed({
  required String privateSeedB64,
  required int points,
  DateTime? now,
}) async {
  final idx = _pointsByIndex.indexOf(points);
  if (idx < 0) {
    throw const LicenseException('points not supported');
  }
  final seed = base64Decode(privateSeedB64.trim());
  if (seed.length != 32) {
    throw const LicenseException('privateSeedB64 长度不正确');
  }
  now?.millisecondsSinceEpoch;
  final kp = await _algo.newKeyPairFromSeed(seed);
  final nonceBytes = _randomBytes(_v3NonceLength);
  final payload = <int>[idx, ...nonceBytes];
  final sig = await _algo.sign(payload, keyPair: kp);
  final bytes = <int>[...payload, ...sig.bytes];

```

```

        final body = _base64UrlEncode(bytes);
        return '$_prefixPointsV3$body';
    }
    static List<int> _randomBytes(int n) {
        final r = Random.secure();
        return List<int>.generate(n, (_) => r.nextInt(256));
    }
    static String _base64UrlEncode(List<int> bytes) {
        return base64Url.encode(bytes).replaceAll('=', '');
    }
    static List<int> _base64UrlDecode(String input) {
        final padded = input + ('=' * ((4 - input.length % 4) % 4));
        try {
            return base64Url.decode(padded);
        } catch (_) {
            throw const LicenseException('卡密内容无法解析');
        }
    }
}

```

```

// File: lib\ai\local_llm\llm_client.dart
import 'dart:async';
import 'dart:io';
import 'package:flutter/foundation.dart';
import 'package:path/path.dart' as p;
import 'mnn_client.dart';
import 'model_manager.dart';
abstract class LlmClient {
    bool get isAvailable;
    String? get currentModel;
    Future<bool> initialize({String? model});
    Future<String> generate({
        required String prompt,
        int maxTokens = 512,
        double temperature = 0.7,
        double topP = 0.9,
        int topK = 40,
        double minP = 0.0,
        double repetitionPenalty = 1.1,
    });
    Stream<String> generateStream({
        required String prompt,
        int maxTokens = 512,
        double temperature = 0.7,
    });
}

```

```

        double topP = 0.9,
        int topK = 40,
        double minP = 0.0,
        double repetitionPenalty = 1.1,
    });
    Future<void> dispose();
}

class LlmClientMnn implements LlmClient {
    final MnnClient _client = MnnClient();
    String? _currentModel;
    @override
    bool get isAvailable => _client.isAvailable;
    @override
    String? get currentModel => _currentModel;
    @override
    Future<bool> initialize({String? model}) async {
        // 如果已经初始化了相同的模型，则跳过
        if (_client.isAvailable && (_currentModel == model || model == null)) {
            debugPrint(
                '[LlmClientMnn] Already initialized with model: $_currentModel');
            return true;
        }
        // 检查平台是否支持 MNN
        final platformAvailable = await MnnClient.isPlatformAvailable();
        if (!platformAvailable) {
            debugPrint('[LlmClientMnn] Platform not available');
            return false;
        }
        // 获取模型路径
        // - model 为空：使用默认模型
        // - model 是绝对路径：直接使用
        // - 否则认为是模型 id：从 ModelManager 获取对应模型目录
        String? modelPath;
        if (model == null) {
            modelPath = await _getDefaultModelPath();
        } else if (p.isAbsolute(model)) {
            modelPath = model;
        } else {
            modelPath = await ModelManager.getModelPath(model);
        }
        if (modelPath == null) {
            debugPrint('[LlmClientMnn] Model path is null');
            return false;
        }
    }
}

```

```

debugPrint('[LlmClientMnn] Initializing with modelPath: $modelPath');
// Debug: Check model files
try {
    final dir = Directory(modelPath);
    debugPrint('[LlmClientMnn] Checking model directory: ${dir.path}');
    if (await dir.exists()) {
        /*
        final files = await dir.list().toList();
        debugPrint('[LlmClientMnn] Model directory contents (${files.length} files):');
        for (final file in files) {
            if (file is File) {
                final size = await file.length();
                debugPrint('  - ${file.path.split('/').last}: $size bytes');
            } else {
                debugPrint('  - ${file.path.split('/').last} (Dir)');
            }
        }
        */
        // Check specific required files
        final requiredFiles = ['config.json', 'llm.mnn', 'tokenizer.txt'];
        for (var f in requiredFiles) {
            final file = File('${dir.path}/${f}');
            if (!await file.exists()) {
                debugPrint('[LlmClientMnn] CRITICAL ERROR: Required file missing: $f');
            } else {
                final size = await file.length();
                if (size == 0) debugPrint('[LlmClientMnn] CRITICAL ERROR: File empty: $f');
            }
        }
    } else {
        debugPrint('[LlmClientMnn] CRITICAL ERROR: Model directory does not exist!');
    }
} catch (e) {
    debugPrint('[LlmClientMnn] Error checking model files: $e');
}
final result = await _client.initialize(modelPath: modelPath);
if (result) {
    _currentModel = model ?? ModelManager.qwen3_0_6b;
    debugPrint('[LlmClientMnn] Initialization successful');
} else {
    debugPrint('[LlmClientMnn] Initialization failed');
}
return result;
}

```



```

Future<String?> _getDefaultModelPath() async {
    final modelPath = await ModelManager.getModelPath(ModelManager.qwen3_0_6b);
    return modelPath;
}

@override
Future<String> generate({
    required String prompt,
    int maxTokens = 512,
    double temperature = 0.7,
    double topP = 0.9,
    int topK = 40,
    double minP = 0.0,
    double repetitionPenalty = 1.1,
}) async {
    final result = await _client.generate(
        prompt: prompt,
        maxTokens: maxTokens,
        temperature: temperature,
        topP: topP,
        topK: topK,
        minP: minP,
        repetitionPenalty: repetitionPenalty,
    );
    if (result == null) {
        throw Exception('MNN generation returned null');
    }
    return result;
}

@override
Stream<String> generateStream({
    required String prompt,
    int maxTokens = 512,
    double temperature = 0.7,
    double topP = 0.9,
    int topK = 40,
    double minP = 0.0,
    double repetitionPenalty = 1.1,
}) async* {
    yield* _client.generateStream(
        prompt: prompt,
        maxTokens: maxTokens,
        temperature: temperature,
        topP: topP,
        topK: topK,
    );
}

```

```

        minP: minP,
        repetitionPenalty: repetitionPenalty,
    );
}
@override
Future<void> dispose() async {
    await _client.dispose();
}
}

/// 创建适合当前平台的本地 LLM 客户端
LlmClient createLocalLlmClient() {
    if (Platform.isIOS || Platform.isAndroid) {
        // iOS 和 Android 使用 MNN
        return LlmClientMnn();
    } else {
        // 其他平台暂不支持
        throw UnsupportedError('Local LLM is only supported on iOS and Android.');
```

```

    }
}

// File: lib\ai\local_llm\mnn_client.dart
```

```

import 'dart:async';
import 'dart:convert';
import 'dart:io';
import 'package:flutter/foundation.dart';
import 'package:flutter/services.dart';

class MnnClient {
    static const MethodChannel _channel = MethodChannel('airread/local_llm');
    static const EventChannel _eventChannel =
        EventChannel('airread/local_llm_stream');
    bool _isInitialized = false;
    String? _modelPath;
    MnnClient();
    bool get isAvailable => _isInitialized;
    String? get modelPath => _modelPath;
    static Future<bool> isPlatformAvailable() async {
        // 桌面端暂时返回 true 以支持开发测试（使用模拟实现或通过插件支持）
        if (!kIsWeb &&
            (Platform.isWindows || Platform.isMacOS || Platform.isLinux)) {
            return true;
        }
        try {
            final isAvailable = await _channel.invokeMethod<bool>('isAvailable');
            if (isAvailable == false && Platform.isAndroid) {
```

```

        final error = await _channel.invokeMethod<String>('getNativeLoadError');
        if (error != null) {
            debugPrint('[MnnClient] MNN not available due to: $error');
        }
    }
    return isAvailable ?? false;
} catch (e) {
    debugPrint('[MnnClient] isPlatformAvailable error: $e');
    return false;
}
}

Future<bool> initialize({required String modelPath}) async {
    try {
        final result = await _channel.invokeMethod<bool>('initialize', {
            'modelPath': modelPath,
        });
        if (result == true) {
            _isInitialized = true;
            _modelPath = modelPath;
            return true;
        } else {
            _isInitialized = false;
            return false;
        }
    } on PlatformException catch (e) {
        debugPrint('[MnnClient] initialize error: ${e.message}');
        _isInitialized = false;
        return false;
    } catch (e) {
        debugPrint('[MnnClient] initialize unexpected error: $e');
        _isInitialized = false;
        return false;
    }
}

Future<String?> generate({
    required String prompt,
    int maxTokens = 512,
    double temperature = 0.7,
    double topP = 0.9,
    int topK = 40,
    double minP = 0.0,
    double presencePenalty = 0.0,
    double repetitionPenalty = 1.1,
    bool enableThinking = false,

```

```

    }) async {
        if (!_isInitialized) {
            throw Exception('MNN client not initialized');
        }
        try {
            final result = await _channel.invokeMethod<String>('chatOnce', {
                'userText': prompt,
                'maxNewTokens': maxTokens,
                'maxInputTokens': 2048,
                'temperature': temperature,
                'topP': topP,
                'topK': topK,
                'minP': minP,
                'presencePenalty': presencePenalty,
                'repetitionPenalty': repetitionPenalty,
                'enableThinking': enableThinking,
            });
            return result;
        } on PlatformException catch (e) {
            debugPrint('[MnnClient] generate error: ${e.message}');
            throw Exception('Failed to generate: ${e.message}');
        }
    }
}

Stream<String> generateStream({
    required String prompt,
    int maxTokens = 512,
    double temperature = 0.7,
    double topP = 0.9,
    int topK = 40,
    double minP = 0.0,
    double presencePenalty = 0.0,
    double repetitionPenalty = 1.1,
    bool enableThinking = false,
}) async* {
    if (!_isInitialized) {
        throw Exception('MNN client not initialized');
    }
    final controller = StreamController<String>();
    final decoderSink = utf8.decoder.startChunkedConversion(controller);
    StreamSubscription? subscription;
    controller.onCancel = () {
        debugPrint('MnnClient: Controller cancelled, cancelling subscription');
        subscription?.cancel();
        decoderSink.close();
    };

```

```

cancel();
};
try {
    // 1. 先监听事件通道，确保 streamSink 已准备好
    subscription = _eventChannel.receiveBroadcastStream().listen(
        (dynamic event) {
            if (event is Map) {
                final type = event['type'] as String?;
                if (type == 'chunk') {
                    final data = event['data'];
                    if (data is String) {
                        if (data.isNotEmpty) {
                            controller.add(data);
                        }
                    } else if (data is Uint8List) {
                        if (data.isNotEmpty) {
                            decoderSink.add(data);
                        }
                    }
                } else if (type == 'done') {
                    debugPrint('MnnClient: Stream done');
                    subscription?.cancel();
                    decoderSink.close();
                } else if (type == 'error') {
                    final msg = event['data'] as String? ?? 'Unknown error';
                    debugPrint('MnnClient: Stream error: $msg');
                    subscription?.cancel();
                    controller.addError(msg);
                    decoderSink.close();
                }
            }
        },
        onError: (dynamic error) {
            debugPrint('MnnClient: EventChannel error: $error');
            subscription?.cancel();
            controller.addError(error);
            decoderSink.close();
        },
        onDone: () {
            debugPrint('MnnClient: EventChannel done');
            decoderSink.close();
        },
    );
    // 2. 再调用 native 启动生成

```

```

        await _channel.invokeMethod('chatStream', {
            'userText': prompt,
            'maxNewTokens': maxTokens,
            'maxInputTokens': 4096,
            'temperature': temperature,
            'topP': topP,
            'topK': topK,
            'minP': minP,
            'presencePenalty': presencePenalty,
            'repetitionPenalty': repetitionPenalty,
            'enableThinking': enableThinking,
        });
        yield* controller.stream;
    } catch (e) {
        debugPrint('MnnClient: invokeMethod error: $e');
        subscription?.cancel();
        controller.close();
        throw Exception('Failed to start stream: $e');
    } finally {
        await subscription?.cancel();
    }
}

Future<void> cancel() async {
    try {
        await _channel.invokeMethod('cancelChatStream');
    } catch (e) {
        debugPrint('[MnnClient] cancel error: $e');
    }
}

Future<String?> dumpConfig() async {
    if (!_isInitialized) {
        return null;
    }
    try {
        final result = await _channel.invokeMethod<String>('dumpConfig');
        return result;
    } catch (e) {
        debugPrint('[MnnClient] dumpConfig error: $e');
        return null;
    }
}

Future<void> dispose() async {
    _isInitialized = false;
    _modelPath = null;
}

```

```
}  
}
```

```
// File: lib\ai\local_llm\mn_model_downloader.dart
```

```
import 'dart:async';
```

```
import 'dart:io';
```

```
import 'package:flutter/foundation.dart';
```

```
import 'package:flutter/services.dart';
```

```
import 'package:http/http.dart' as http;
```

```
import 'package:http/io_client.dart';
```

```
import 'package:path_provider/path_provider.dart';
```

```
import 'package:path/path.dart' as p;
```

```
import 'mn_model_spec.dart';
```

```
/// MNN 模型下载状态
```

```
enum ModelDownloadStatus {
```

```
    notDownloaded,
```

```
    downloading,
```

```
    extracting,
```

```
    completed,
```

```
    failed,
```

```
}
```

```
/// MNN 模型下载器
```

```
/// 从 ModelScope 下载 MNN 模型（直接下载文件）
```

```
class MnnModelDownloader {
```

```
    final MnnModelSpec spec;
```

```
    MnnModelDownloader({required this.spec});
```

```
    int get estimatedTotalSize => spec.estimatedTotalSizeBytes;
```

```
    // 下载状态流
```

```
    final _statusController = StreamController<ModelDownloadStatus>.broadcast();
```

```
    final _progressController = StreamController<double>.broadcast();
```

```
    final _currentFileController = StreamController<String>.broadcast();
```

```
    Stream<ModelDownloadStatus> get statusStream => _statusController.stream;
```

```
    Stream<double> get progressStream => _progressController.stream;
```

```
    Stream<String> get currentFileStream => _currentFileController.stream;
```

```
    ModelDownloadStatus _status = ModelDownloadStatus.notDownloaded;
```

```
    double _progress = 0.0;
```

```
    String _currentFile = "";
```

```
    http.Client? _httpClient;
```

```
    bool _cancelled = false;
```

```
    bool _sessionPrepared = false;
```

```
    final Map<String, int> _receivedBytesByFile = {};
```

```
    final Map<String, int> _expectedBytesByFile = {};
```

```
    ModelDownloadStatus get status => _status;
```

```
    double get progress => _progress;
```

```

String get currentFile => _currentFile;
static const int _defaultMaxAttemptsPerFile = 6;
static const Duration _defaultInitialBackoff = Duration(milliseconds: 800);
static const Duration _defaultMaxBackoff = Duration(seconds: 30);
static const Duration _defaultRequestTimeout = Duration(minutes: 20);
static const Duration _defaultSmallFileTimeout = Duration(seconds: 45);
static const Duration _defaultStreamInactivityTimeoutSmall =
    Duration(seconds: 20);
static const Duration _defaultStreamInactivityTimeoutLarge =
    Duration(seconds: 45);
String _lastError = '';
Future<Directory> _getAppDirSafe() async {
    if (kIsWeb) {
        throw UnsupportedError('Local model storage is not supported on Web.');
```

```
    }
```

```
    try {
```

```
        return await getApplicationDocumentsDirectory();
```

```
    } on MissingPluginException {
```

```
        return Directory.systemTemp;
```

```
    } catch (_) {
```

```
        return Directory.systemTemp;
```

```
    }
```

```
}
```

```
/// 获取模型目录路径
```

```
Future<String> getModelDir() async {
```

```
    final appDir = await _getAppDirSafe();
```

```
    return p.join(appDir.path, spec.modelDirRelative);
```

```
}
```

```
/// 检查模型是否已下载
```

```
Future<bool> isModelDownloaded() async {
```

```
    try {
```

```
        final modelDir = await getModelDir();
```

```
        final dir = Directory(modelDir);
```

```
        if (!await dir.exists()) {
```

```
            return false;
```

```
        }
```

```
        // 检查所有必需文件是否存在
```

```
        for (final fileName in spec.criticalFiles) {
```

```
            final filePath = p.join(modelDir, fileName);
```

```
            final f = File(filePath);
```

```
            if (!await f.exists()) {
```

```
                debugPrint('[MnnModelDownloader] Missing critical file: $fileName');
```

```
                return false;
```

```
            }
```



```

        final minBytes = _minExpectedBytes(fileName);
        if (minBytes != null) {
            final size = await f.length();
            if (size < minBytes) {
                debugPrint('[MnnModelDownloader] Critical file too small: $fileName size=$size
min=$minBytes');
                return false;
            }
        }
    }
}

// 检查 config.json 是否存在
// 但为了 isModelInstalled 返回 true, 我们需要确保 config.json 存在
final configPath = p.join(modelDir, 'config.json');
if (!await File(configPath).exists()) {
    // 尝试修复: 如果 llm_config.json 存在, 复制一份
    final llmConfigPath = p.join(modelDir, 'llm_config.json');
    if (await File(llmConfigPath).exists()) {
        try {
            await File(llmConfigPath).copy(configPath);
            debugPrint('[MnnModelDownloader] Auto-created config.json from
llm_config.json');
        } catch (e) {
            debugPrint('[MnnModelDownloader] Failed to create config.json: $e');
            return false;
        }
    } else {
        debugPrint('[MnnModelDownloader] Missing config.json and llm_config.json');
        return false;
    }
}
return true;
} catch (e) {
    debugPrint('[MnnModelDownloader] Error checking model: $e');
    return false;
}
}

/// 获取已下载的文件大小
Future<int> getDownloadedSize() async {
    try {
        final modelDir = await getModelDir();
        int totalSize = 0;
        for (final fileName in spec.filesToDownload) {
            final filePath = p.join(modelDir, fileName);
            final file = File(filePath);

```

```

        if (await file.exists()) {
            totalSize += await file.length();
        }
    }
    return totalSize;
} catch (e) {
    return 0;
}
}

/// 开始下载模型
Future<bool> download() async {
    if (_status == ModelDownloadStatus.downloading ||
        _status == ModelDownloadStatus.extracting) {
        return false;
    }
    _status = ModelDownloadStatus.downloading;
    _statusController.add(_status);
    _cancelled = false;
    _httpClient = IOClient(_createHttpClient());
    _sessionPrepared = false;
    _receivedBytesByFile.clear();
    _expectedBytesByFile.clear();
    try {
        final modelDir = await getModelDir();
        // 创建目录
        final dir = Directory(modelDir);
        if (!await dir.exists()) {
            await dir.create(recursive: true);
        }
        await _prepareModelScopeSession();
        await _initializeProgressBaseline(modelDir);
        // 逐个下载文件
        for (final fileName in spec.filesToDownload) {
            if (_cancelled) break;
            _currentFile = fileName;
            _currentFileController.add(_currentFile);
            final filePath = p.join(modelDir, fileName);
            if (!_supportsResume(fileName)) {
                _receivedBytesByFile[fileName] = 0;
                _expectedBytesByFile[fileName] = 0;
                _updateOverallProgress();
            }
            final success = await _downloadSingleFileWithRetry(
                fileName,

```

```

        filePath,
        (bytes) {
            _receivedBytesByFile[fileName] = bytes;
            _updateOverallProgress();
        }
    );
    if (!success) {
        _status = ModelDownloadStatus.failed;
        _statusController.add(_status);
        return false;
    }
}
if (_cancelled) {
    _status = ModelDownloadStatus.notDownloaded;
    _statusController.add(_status);
    return false;
}
_status = ModelDownloadStatus.completed;
_statusController.add(_status);
_progress = 1.0;
_progressController.add(1.0);
return true;
} catch (e) {
    debugPrint('[MnnModelDownloader] Download error: $e');
    _status = ModelDownloadStatus.failed;
    _statusController.add(_status);
    return false;
} finally {
    _httpClient?.close();
    _httpClient = null;
}
}
Future<void> _initializeProgressBaseline(String modelDir) async {
    for (final fileName in spec.filesToDownload) {
        final filePath = p.join(modelDir, fileName);
        if (_supportsResume(fileName)) {
            _receivedBytesByFile[fileName] = await _safeFileLength(filePath);
        } else {
            _receivedBytesByFile[fileName] = 0;
        }
        _expectedBytesByFile[fileName] = 0;
    }
    _updateOverallProgress();
}

```

```

void _updateOverallProgress() {
    final files = spec.filesToDownload;
    if (files.isEmpty) return;
    // Calculate total received bytes
    int totalReceived = 0;
    for (final fileName in files) {
        totalReceived += _receivedBytesByFile[fileName] ?? 0;
    }
    // Use estimated total size as the baseline
    // If estimated size is 0 or invalid, fallback to a small number to avoid div by zero
    final totalExpected = spec.estimatedTotalSizeBytes > 0
        ? spec.estimatedTotalSizeBytes
        : 1024 * 1024 * 100; // 100MB fallback
    double p = totalReceived / totalExpected;
    // Clamp to 0.0 - 0.99
    // We leave 1.0 for the explicit completion state
    final next = p.clamp(0.0, 0.99);
    if ((next - _progress).abs() > 0.001) {
        _progress = next;
        _progressController.add(_progress);
    }
}

HttpClient _createHttpClient() {
    final client = HttpClient();
    client.connectionTimeout = const Duration(seconds: 30);
    return client;
}

Future<void> _prepareModelScopeSession() async {
    if (_sessionPrepared) return;
    final client = _httpClient;
    if (client == null) return;
    final baseUrl = Uri.parse(spec.baseUrl);
    final warmupUri = Uri(
        scheme: baseUrl.scheme,
        host: baseUrl.host,
        path: '/',
    );
    try {
        final request = http.Request('GET', warmupUri);
        _applyDefaultHeaders(request, baseUrl: baseUrl);
        final response = await client.send(request).timeout(
            const Duration(seconds: 20),
        );
        await response.stream.drain<void>();
    }
}

```

```

        debugPrint(
            '[MnnModelDownloader] Session warmup status=${response.statusCode}',
        );
    } catch (e) {
        debugPrint('[MnnModelDownloader] Session warmup failed: $e');
    } finally {
        _sessionPrepared = true;
    }
}

void _applyDefaultHeaders(
    http.BaseRequest request, {
    required Uri baseUri,
}) {
    request.headers['User-Agent'] =
        'Mozilla/5.0 (iPhone; CPU iPhone OS 17_0 like Mac OS X) AppleWebKit/605.1.15
(KHTML, like Gecko) Version/17.0 Mobile/15E148 Safari/604.1';
    request.headers['Accept'] =
        'text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8';
    request.headers['Accept-Language'] = 'zh-CN,zh;q=0.9,en;q=0.8';
    request.headers['Connection'] = 'keep-alive';
    final referer = _modelScopeReferer(baseUri);
    if (referer != null) request.headers['Referer'] = referer;
}

String? _modelScopeReferer(Uri baseUri) {
    final seg = baseUri.pathSegments.where((e) => e.isNotEmpty).toList();
    final modelsIndex = seg.indexOf('models');
    if (modelsIndex < 0 || seg.length <= modelsIndex + 2) return null;
    final org = seg[modelsIndex + 1];
    final repo = seg[modelsIndex + 2];
    return Uri(
        scheme: baseUri.scheme,
        host: baseUri.host,
        path: '/models/$org/$repo/summary',
    ).toString();
}

Future<bool> _downloadSingleFileWithRetry(
    String fileName,
    String savePath,
    Function(int receivedTotal) onProgress,
) async {
    int attempt = 0;
    Duration backoff = _defaultInitialBackoff;
    while (!_cancelled) {
        attempt++;

```

```

    final existingBytes =
        _supportsResume(fileName) ? await _safeFileLength(savePath) : 0;
    final res = await _downloadSingleFileOnce(
        fileName: fileName,
        savePath: savePath,
        existingBytes: existingBytes,
        onProgress: onProgress,
    );
    if (res == _DownloadResult.success) return true;
    if (res == _DownloadResult.cancelled) return false;
    if (res == _DownloadResult.fatal) return false;
    debugPrint(
        '[MnnModelDownloader] Retryable failure for $fileName (attempt=$attempt)
error=$_lastError',
    );
    if (_defaultMaxAttemptsPerFile > 0 && attempt >= _defaultMaxAttemptsPerFile) {
        debugPrint(
            '[MnnModelDownloader] Giving up on $fileName after $attempt attempts.
lastError=$_lastError',
        );
        return false;
    }
    await Future<void>.delayed(backoff + _jitter(backoff));
    backoff = _nextBackoff(backoff);
}
return false;
}

Duration _nextBackoff(Duration current) {
    final nextMs = (current.inMilliseconds * 2).clamp(
        _defaultInitialBackoff.inMilliseconds,
        _defaultMaxBackoff.inMilliseconds,
    );
    return Duration(milliseconds: nextMs);
}

Duration _jitter(Duration base) {
    final ms = base.inMilliseconds;
    if (ms <= 0) return Duration.zero;
    final jitter = (ms * 0.2).round();
    final now = DateTime.now().microsecondsSinceEpoch;
    final r = (now % (jitter * 2 + 1)) - jitter;
    return Duration(milliseconds: r);
}

static Future<int> _safeFileLength(String path) async {
    try {

```

```

        final f = File(path);
        if (!await f.exists()) return 0;
        return await f.length();
    } catch (_) {
        return 0;
    }
}

/// 下载单个文件
Future<_DownloadResult> _downloadSingleFileOnce({
    required String fileName,
    required String savePath,
    required int existingBytes,
    required Function(int receivedTotal) onProgress,
}) async {
    _lastError = "";
    final client = _httpClient;
    if (client == null) {
        _lastError = 'http client is null';
        return _DownloadResult.retryable;
    }
    http.StreamedResponse? response;
    Object? lastException;
    for (final url in _candidateUrlsForFile(fileName)) {
        debugPrint('[MnnModelDownloader] Downloading $fileName from $url');
        try {
            final request = http.Request('GET', Uri.parse(url));
            _applyDefaultHeaders(request, baseUrl: Uri.parse(spec.baseUrl));
            request.headers['Accept'] = 'application/octet-stream, */*;q=0.8';
            if (existingBytes > 0 && _supportsResume(fileName)) {
                request.headers['Range'] = 'bytes=$existingBytes-';
            }
            response = await client.send(request).timeout(
                _requestTimeoutForFile(fileName),
                onTimeout: () {
                    throw TimeoutException('Download timeout for $fileName');
                },
            );
        } catch (e) {
            if (_cancelled) return _DownloadResult.cancelled;
            lastException = e;
            _lastError = e.toString();
            if (_isRetryableException(e)) return _DownloadResult.retryable;
            return _DownloadResult.fatal;
        }
    }
}

```

```

        if (_cancelled) return _DownloadResult.cancelled;
        if (response.statusCode == 200 || response.statusCode == 206) {
            break;
        }
        // Handle 416 Range Not Satisfiable
        // Usually means the file is already fully downloaded (existingBytes >= remoteSize)
        if (response.statusCode == 416) {
            debugPrint(
                '[MnnModelDownloader] 416 Range Not Satisfiable for $fileName. Checking local
file...');
            final localSize = await _safeFileLength(savePath);
            final minBytes = _minExpectedBytes(fileName) ?? 1;
            if (localSize > 0 && localSize >= minBytes) {
                debugPrint(
                    '[MnnModelDownloader] Local file seems valid (size=$localSize). Treating as
success.');
```

onProgress(localSize); // Update progress with full size

```

                return _DownloadResult.success;
            } else {
                debugPrint(
                    '[MnnModelDownloader] Local file invalid/small ($localSize). Deleting and
retrying.');
```

try {

```

                    await File(savePath).delete();
                } catch (_) {}
                _lastError = '416 but local file invalid';
                // Continue to next attempt (which will be from scratch since file is deleted)
                continue;
            }
        }
        debugPrint(
            '[MnnModelDownloader] Failed to download $fileName: ${response.statusCode}',
        );
        _lastError = 'http ${response.statusCode}';
        if (_isRetryableStatusCode(response.statusCode)) return _DownloadResult.retryable;
        final shouldTryNext = response.statusCode == 404 || response.statusCode == 403;
        if (!shouldTryNext) return _DownloadResult.fatal;
    }
    if (response == null) {
        _lastError = lastException?.toString() ?? 'no response';
        return _DownloadResult.fatal;
    }
    if (_cancelled) return _DownloadResult.cancelled;
    if (response.statusCode != 200 && response.statusCode != 206) {

```



```

        debugPrint('[MnnModelDownloader]      Failed      to      download      $fileName:
${response.statusCode}');
        _lastError = 'http ${response.statusCode}';
        if (_isRetryableStatusCode(response.statusCode)) return _DownloadResult.retryable;
        return _DownloadResult.fatal;
    }
    final file = File(savePath);
    int startOffset = 0;
    bool append = false;
    if (response.statusCode == 206 && existingBytes > 0) {
        startOffset = existingBytes;
        append = true;
    } else if (existingBytes > 0 && response.statusCode == 200) {
        try {
            await file.writeAsBytes(const [], mode: FileMode.write);
        } catch (_) {}
        startOffset = 0;
        append = false;
    }
    final totalBytes = _totalBytesFromResponse(response);
    final expected = totalBytes ??
        response.contentLength ??
        spec.minExpectedBytesByFile[fileName] ??
        1;
    _expectedBytesByFile[fileName] = expected;
    _updateOverallProgress();
    final sink = file.openWrite(mode: append ? FileMode.append : FileMode.write);
    int receivedBytes = 0;
    try {
        await for (final chunk
            in response.stream.timeout(_streamInactivityTimeoutForFile(fileName))) {
            if (_cancelled) {
                await sink.close();
                return _DownloadResult.cancelled;
            }
            sink.add(chunk);
            receivedBytes += chunk.length;
            onProgress(startOffset + receivedBytes);
        }
        await sink.close();
    } catch (e) {
        try {
            await sink.close();
        } catch (_) {}
    }

```

```

        if (_cancelled) return _DownloadResult.cancelled;
        _lastError = e.toString();
        if (_isRetryableException(e)) return _DownloadResult.retryable;
        return _DownloadResult.retryable;
    }
    final finalSize = await _safeFileLength(savePath);
    if (totalBytes != null && totalBytes > 0 && finalSize != totalBytes) {
        debugPrint('[MnnModelDownloader] Incomplete file for $fileName: size=$finalSize
total=$totalBytes');
        _lastError = 'incomplete file size=$finalSize total=$totalBytes';
        return _DownloadResult.retryable;
    }
    final looksTextPointer = await _looksLikeTextPointer(file);
    if (looksTextPointer) {
        debugPrint('[MnnModelDownloader] Downloaded content looks like a text pointer/html
for $fileName');
        try {
            await file.delete();
        } catch (_) {}
        _lastError = 'downloaded content looks like html/pointer';
        return _DownloadResult.fatal;
    }
    final minBytes = _minExpectedBytes(fileName);
    if (minBytes != null && finalSize < minBytes) {
        debugPrint('[MnnModelDownloader] File too small for $fileName: size=$finalSize
minExpected=$minBytes');
        _lastError = 'file too small size=$finalSize minExpected=$minBytes';
        return _DownloadResult.retryable;
    }
    debugPrint('[MnnModelDownloader] $fileName downloaded successfully');
    return _DownloadResult.success;
}

Duration _requestTimeoutForFile(String fileName) {
    final isLarge = fileName == spec.progressFileName || fileName.endsWith('.weight');
    return isLarge ? _defaultRequestTimeout : _defaultSmallFileTimeout;
}

bool _supportsResume(String fileName) {
    return fileName == spec.progressFileName || fileName.endsWith('.weight');
}

Duration _streamInactivityTimeoutForFile(String fileName) {
    final isLarge = fileName == spec.progressFileName || fileName.endsWith('.weight');
    return isLarge
        ? _defaultStreamInactivityTimeoutLarge
        : _defaultStreamInactivityTimeoutSmall;
}

```

```

}
List<String> _candidateUrlsForFile(String fileName) {
    final baseUri = Uri.parse(spec.baseUri);
    final base = baseUri.resolve(fileName).toString();
    final candidates = <String>[base];
    void addIf(String url) {
        if (!candidates.contains(url)) candidates.add(url);
    }
    for (final url in List<String>.from(candidates)) {
        if (url.contains('/resolve/master/')) {
            addIf(url.replaceFirst('/resolve/master/', '/resolve/main/'));
        }
        if (url.contains('/:modelscope.cn/')) {
            addIf(url.replaceFirst('/:modelscope.cn/', '://www.modelscope.cn/'));
        }
        if (url.contains('/:www.modelscope.cn/')) {
            addIf(url.replaceFirst('/:www.modelscope.cn/', '://modelscope.cn/'));
        }
    }
    return candidates;
}

static bool _isRetryableStatusCode(int code) {
    if (code == 408) return true;
    if (code == 429) return true;
    if (code >= 500 && code <= 599) return true;
    return false;
}

static bool _isRetryableException(Object e) {
    if (e is SocketException) return true;
    if (e is TimeoutException) return true;
    if (e is HandshakeException) return true;
    if (e is HttpException) return true;
    if (e is http.ClientException) return true;
    return false;
}

static int? _totalBytesFromResponse(http.StreamedResponse response) {
    if (response.statusCode == 200) return response.contentLength;
    final contentRange = response.headers['content-range'] ??
response.headers['Content-Range'];
    if (contentRange == null) return null;
    return parseTotalBytesFromContentRange(contentRange);
}

static int? parseTotalBytesFromContentRange(String value) {
    final v = value.trim();

```

```

        final slash = v.lastIndexOf('/');
        if (slash < 0) return null;
        final totalStr = v.substring(slash + 1).trim();
        if (totalStr == '*' || totalStr.isEmpty) return null;
        return int.tryParse(totalStr);
    }
    static Future<bool> _looksLikeTextPointer(File file) async {
        try {
            final raf = await file.open();
            final bytes = await raf.read(512);
            await raf.close();
            if (bytes.isEmpty) return true;
            final text = String.fromCharCode(bytes).toLowerCase();
            if (text.contains('git-lfs.github.com')) return true;
            if (text.contains('<html') || text.contains('<!doctype html')) return true;
            if (text.contains('access denied') || text.contains('forbidden')) return true;
            return false;
        } catch (_) {
            return false;
        }
    }
    int? _minExpectedBytes(String fileName) {
        return spec.minExpectedBytesByFile[fileName];
    }
    /// 取消下载
    void cancel() {
        _cancelled = true;
        _httpClient?.close();
    }
    /// 删除已下载模型
    Future<bool> deleteModel() async {
        try {
            final modelDir = await getModelDir();
            final dir = Directory(modelDir);
            if (await dir.exists()) {
                await dir.delete(recursive: true);
                debugPrint('[MnnModelDownloader] Model deleted');
            }
            return true;
        } catch (e) {
            debugPrint('[MnnModelDownloader] Error deleting model: $e');
            return false;
        }
    }
}

```

```

/// 获取模型路径
Future<String?> getModelPath() async {
  if (await isModelDownloaded()) {
    return getModelDir();
  }
  return null;
}

void dispose() {
  _statusController.close();
  _progressController.close();
  _currentFileController.close();
}

enum _DownloadResult {
  success,
  retryable,
  fatal,
  cancelled,
}

// File: lib\ai\local_llm\mn_model_spec.dart
class MnnModelSpec {
  final String id;
  final String displayName;
  final String sizeLabel;
  final int estimatedTotalSizeBytes;
  final String baseUrl;
  final List<String> filesToDownload;
  final List<String> criticalFiles;
  final Map<String, int> minExpectedBytesByFile;
  final String progressFileName;
  const MnnModelSpec({
    required this.id,
    required this.displayName,
    required this.sizeLabel,
    required this.estimatedTotalSizeBytes,
    required this.baseUrl,
    required this.filesToDownload,
    required this.criticalFiles,
    required this.minExpectedBytesByFile,
    this.progressFileName = 'llm.mnn.weight',
  });
  String get modelDirRelative => 'models/$id';
}

```

```

// File: lib\ai\local_llm\model_manager.dart
import 'mnn_model_downloader.dart';
import 'mnn_model_spec.dart';
/// 模型管理器
/// 负责检查、下载和管理 MNN 模型
class ModelManager {
  static const String qwen3_0_6b = 'qwen3-0.6b-mnn';
  static const MnnModelSpec qwen3Spec = MnnModelSpec(
    id: qwen3_0_6b,
    displayName: 'Qwen3-0.6B',
    sizeLabel: '450M',
    estimatedTotalSizeBytes: 455 * 1024 * 1024,
    baseUrl: 'https://modelscope.cn/models/MNN/Qwen3-0.6B-MNN/resolve/master/',
    filesToDownload: [
      'config.json',
      'llm_config.json',
      'llm.mnn',
      'llm.mnn.weight',
      'tokenizer.txt',
    ],
    criticalFiles: [
      'llm.mnn',
      'llm.mnn.weight',
      'tokenizer.txt',
      'config.json',
    ],
    minExpectedBytesByFile: {
      'llm.mnn.weight': 100 * 1024 * 1024,
      'llm.mnn': 200 * 1024,
      'tokenizer.txt': 4 * 1024,
      'config.json': 200,
      'llm_config.json': 200,
    },
  );
  static const List<MnnModelSpec> localModels = [qwen3Spec];
  static MnnModelSpec specFor(String modelId) {
    return localModels.firstWhere((e) => e.id == modelId, orElse: () => qwen3Spec);
  }
  static String displayNameFor(String modelId) => specFor(modelId).displayName;
  static String sizeLabelFor(String modelId) => specFor(modelId).sizeLabel;
  /// 检查模型是否已安装到文档目录
  static Future<bool> isModelInstalled(String modelId) async {
    return await MnnModelDownloader(spec: specFor(modelId)).isModelDownloaded();
  }
}

```

```
// ... [中间代码省略] ...
```

```
const SizedBox(width: 6),  
Icon(  
  m.reasoningCollapsed  
    ? Icons.expand_more  
    : Icons.expand_less,  
  size: 18,  
  color: widget.textColor  
    .withOpacityCompat(0.55),  
),  
],  
),  
if (!m.reasoningCollapsed) ...[  
  const SizedBox(height: 8),  
  Container(  
    width: double.infinity,  
    padding:  
      const EdgeInsets.symmetric(  
        horizontal: 10,  
        vertical: 10),  
    decoration: BoxDecoration(  
      color: widget.textColor  
        .withOpacityCompat(  
          widget.isDark  
            ? 0.06  
            : 0.035),  
      borderRadius:  
        BorderRadius.circular(12),  
    ),  
    child: ConstrainedBox(  
      constraints:  
        const BoxConstraints(  
          maxHeight: 140),  
      child: SingleChildScrollView(  
        reverse: true,  
        child: Text(  
          m.reasoning.trim(),  
          style: TextStyle(  
            color: widget.textColor
```

```

        .withOpacityCompat(
            0.78),
        height: 1.35,
        fontSize: 13,
    ),
    ),
    ),
    ),
    const SizedBox(height: 10),
],
],
Text(
    m.text.isEmpty &&
        _activeReplyIndex == i &&
        m.reasoning.trim().isEmpty
        ? '...'
        : m.text,
    style: bubbleTextStyle,
),
],
),
),
);
Widget child;
if (!showActions) {
    child = bubble;
} else if (placeActionsBelow) {
    child = Column(
        mainAxisAlignment: MainAxisAlignment.min,
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
            bubble,
            Padding(
                padding: const EdgeInsets.only(bottom: 6),
                child: actionButtons,
            ),
        ],
    );
} else {
    final side = Padding(
        padding: const EdgeInsets.only(bottom: 6),
        child: actionButtons,
    );

```



```

        child = Row(
          mainAxisAlignment: MainAxisAlignment.min,
          crossAxisAlignment: CrossAxisAlignment.end,
          children: isUser
            ? [side, const SizedBox(width: 6), bubble]
            : [bubble, const SizedBox(width: 6), side],
        );
      }
      return Align(
        key: ValueKey('qa_msg_${i}_${m.text.hashCode}'),
        alignment: align,
        child: child,
      );
    },
  );
},
),
),
if (_messageState == _MessageState.thinking &&
    (_activeReplyIndex == null ||
      (_activeReplyIndex! >= 0 &&
        _activeReplyIndex! < _messages.length &&
        _messages[_activeReplyIndex!].text.trim().isEmpty &&
        _messages[_activeReplyIndex!]
          .reasoning
          .trim()
          .isEmpty)))
    Align(
      alignment: Alignment.centerLeft,
      child: Padding(
        padding: const EdgeInsets.symmetric(vertical: 8.0),
        child: Text('思考中...',
          style: TextStyle(
            color: widget.textColor.withOpacityCompat(0.6),
            fontSize: 15)),
      ),
    ),
const SizedBox(height: 8),
Wrap(
  spacing: 10,
  runSpacing: 10,
  children: [
    actionChip(
      label: '新话题',

```

```

        onTap: qaBlocked || _messageState != _MessageState.idle
          ? null
          : _startNewTopic,
      ),
      actionChip(
        label: '总结本章',
        onTap: qaBlocked || _messageState != _MessageState.idle
          ? null
          : () => _sendQuickAction(QAType.summary),
      ),
      actionChip(
        label: '提取要点',
        onTap: qaBlocked || _messageState != _MessageState.idle
          ? null
          : () => _sendQuickAction(QAType.keyPoints),
      ),
    ],
  ),
  const SizedBox(height: 10),
  Row(
    children: [
      Expanded(
        child: Focus(
          onKeyEvent: (node, event) {
            if (event is KeyDownEvent &&
              event.logicalKey == LogicalKeyboardKey.enter &&
              !ServicesBinding
                .instance.keyboard.logicalKeysPressed
                .contains(LogicalKeyboardKey.shiftLeft) &&
              !ServicesBinding
                .instance.keyboard.logicalKeysPressed
                .contains(LogicalKeyboardKey.shiftRight)) {
              if (!qaBlocked &&
                _messageState == _MessageState.idle) {
                _send();
              }
              return KeyEventResult.handled;
            }
            return KeyEventResult.ignored;
          },
          child: TextField(
            controller: _inputCtl,
            enabled: !qaBlocked,
            minLines: 1,

```

一致且清爽

```
maxLines: 4,
textInputAction: TextInputAction.send,
style: const TextStyle(fontSize: 14),
decoration: InputDecoration(
  isDense: true,
  filled: true,
  fillColor: Colors.white,
  hintText: '输入你的问题...',
  contentPadding: const EdgeInsets.symmetric(
    horizontal: 12, vertical: 12),
  border: OutlineInputBorder(
    borderRadius: BorderRadius.circular(12)),
),
contextMenuBuilder: (context, editableTextState) {
  final List<ContextMenuButtonItem> buttonItems =
    editableTextState.contextMenuButtonItems;
  // 仅保留基础编辑功能 (复制/粘贴/剪切/全选), 确保双端

  buttonItems.removeWhere(
    (ContextMenuButtonItem buttonItem) {
      return buttonItem.type !=
        ContextMenuButtonType.cut &&
        buttonItem.type !=
        ContextMenuButtonType.copy &&
        buttonItem.type !=
        ContextMenuButtonType.paste &&
        buttonItem.type !=
        ContextMenuButtonType.selectAll;
    });
  return AdaptiveTextSelectionToolbar.buttonItems(
    anchors: editableTextState.contextMenuAnchors,
    buttonItems: buttonItems,
  );
},
onSubmitted: (_) {
  if (!qaBlocked &&
    _messageState == _MessageState.idle) {
    _send();
  }
},
),
),
const SizedBox(width: 10),
```

```

ElevatedButton(
  onPressed:
    (!qaBlocked && _messageState == _MessageState.idle)
      ? _send
      : null,
  style: ElevatedButton.styleFrom(
    backgroundColor: AppColors.techBlue,
    foregroundColor: Colors.white,
    elevation: 0,
    padding: const EdgeInsets.symmetric(
      horizontal: 14, vertical: 12),
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(12),
    ),
    disabledBackgroundColor:
      AppColors.techBlue.withOpacityCompat(0.45),
    disabledForegroundColor:
      Colors.white.withOpacityCompat(0.75),
  ),
  child: const Text('发送'),
),
],
),
],
),
if (_qaToastText.trim().isEmpty)
  Center(
    child: IgnorePointer(
      child: AnimatedOpacity(
        opacity: _qaToastText.trim().isEmpty ? 1 : 0,
        duration: const Duration(milliseconds: 160),
        child: Container(
          padding: const EdgeInsets.symmetric(
            horizontal: 14, vertical: 8),
          decoration: BoxDecoration(
            color: Colors.black.withOpacityCompat(0.72),
            borderRadius: BorderRadius.circular(20),
          ),
        ),
        child: Text(
          _qaToastText,
          style: const TextStyle(
            color: Colors.white,
            fontSize: 13,
            height: 1.1,

```

```

        decoration: TextDecoration.none,
      ),
      textAlign: TextAlign.center,
    ),
  ),
),
],
),
),
);
return Padding(
  padding:
    EdgeInsets.only(bottom: MediaQuery.of(context).viewInsets.bottom),
  child: GestureDetector(
    onTap: () {
      FocusScope.of(context).unfocus();
      _hideBubbleActions();
    },
    behavior: HitTestBehavior.translucent,
    child: panelContent,
  ),
);
}
}

```

// File: lib\presentation\widgets\air_flow_system.dart

```

import 'dart:math';
import 'package:flutter/material.dart';
class WindConfig {
  double density; // 0.5 to 2.0
  double sizeScale; // 0.8 to 1.5
  double speedScale; // 0.5 to 2.0
  WindConfig({
    this.density = 1.0,
    this.sizeScale = 1.0,
    this.speedScale = 1.0,
  });
}
class WindStream {
  double x;
  double y;
  final double baseSpeed;

```

```

final double baseWidth;
final double baseLength;
final double opacity;
double progress; // 0.0 to 1.0
// Dynamic properties based on config
double currentSpeed = 0;
double currentWidth = 0;
double currentLength = 0;
WindStream({
    required this.x,
    required this.y,
    required this.baseSpeed,
    required this.baseWidth,
    required this.baseLength,
    required this.opacity,
    this.progress = 0.0,
});
void update(WindConfig config) {
    currentSpeed = baseSpeed * config.speedScale;
    currentWidth = baseWidth * config.sizeScale;
    currentLength = baseLength * config.sizeScale;
    y -= currentSpeed;
    progress += 0.01 * config.speedScale;
}
}
class AirFlowController extends ChangeNotifier {
    final List<WindStream> streams = [];
    final Random _random = Random();
    Size canvasSize;
    WindConfig config;
    AirFlowController({
        required this.canvasSize,
        WindConfig? config,
    }) : config = config ?? WindConfig();
    void updateSize(Size newSize) {
        canvasSize = newSize;
        notifyListeners();
    }
    void updateConfig({double? density, double? sizeScale, double? speedScale}) {
        if (density != null) config.density = density;
        if (sizeScale != null) config.sizeScale = sizeScale;
        if (speedScale != null) config.speedScale = speedScale;
        notifyListeners();
    }
}

```

```

void spawnStream() {
  // Strict Limit: Only 1 stream at a time as requested
  if (streams.isNotEmpty) return;
  final startX = _random.nextDouble() * canvasSize.width;
  final startY = canvasSize.height + 200; // Start further below
  streams.add(WindStream(
    x: startX,
    y: startY,
    baseSpeed: 2.0 + _random.nextDouble() * 3.0,
    baseWidth: 30 + _random.nextDouble() * 20,
    baseLength: 200 + _random.nextDouble() * 300,
    opacity: 0.15 + _random.nextDouble() * 0.2, // Reduced visual intensity (opacity)
  ));
}

void updateParticles() {
  for (var i = streams.length - 1; i >= 0; i--) {
    streams[i].update(config);
    // Remove if completely off screen (accounting for length)
    if (streams[i].y < -streams[i].currentLength) {
      streams.removeAt(i);
    }
  }
  // Spawn new streams with reduced frequency
  // 1% chance per frame (approx once every 2 seconds at 60fps)
  if (_random.nextDouble() < 0.01) {
    spawnStream();
  }
  notifyListeners();
}
}

```

```

// File: lib\presentation\widgets\air_title.dart
import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';
import '../core/theme/app_colors.dart';
class AirTitle extends StatefulWidget {
  const AirTitle({super.key});
  @override
  State<AirTitle> createState() => _AirTitleState();
}

class _AirTitleState extends State<AirTitle> with SingleTickerProviderStateMixin {
  late final AnimationController _controller;
  bool _settled = false;
  @override

```

```

void initState() {
  super.initState();
  _controller = AnimationController(
    vsync: this,
    duration: const Duration(milliseconds: 3200),
  );
  WidgetsBinding.instance.addPostFrameCallback(_) {
    if (!mounted) return;
    _controller.forward(from: 0).whenComplete(() {
      if (!mounted) return;
      setState(() {
        _settled = true;
      });
    });
  });
}

@override
void dispose() {
  _controller.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  final theme = Theme.of(context);
  final scheme = theme.colorScheme;
  final isDark = theme.brightness == Brightness.dark;
  final onSurface = scheme.onSurface;
  final baseA = isDark ? AppColors.neonCyan : AppColors.techBlue;
  final baseB = isDark ? AppColors.techBlue : AppColors.neonCyan;
  const titleText = '灵阙';
  return AnimatedBuilder(
    animation: _controller,
    builder: (context, _) {
      final titleGradient = _settled
        ? LinearGradient(
            colors: [
              baseA,
              Color.lerp(baseA, baseB, 0.45) ?? baseA,
              baseB,
            ],
            stops: const [0.0, 0.5, 1.0],
            begin: Alignment.centerLeft,
            end: Alignment.centerRight,
          )

```



```

: () {
  final c = _controller.value;
  const w = 0.26;
  final s0 = (c - w).clamp(0.0, 1.0);
  final s1 = c.clamp(0.0, 1.0);
  final s2 = (c + w).clamp(0.0, 1.0);
  return LinearGradient(
    colors: [
      baseA,
      Color.lerp(baseA, Colors.white, isDark ? 0.26 : 0.22) ??
      baseA,
      baseB,
    ],
    stops: [s0, s1, s2],
    begin: Alignment.centerLeft,
    end: Alignment.centerRight,
  );
}());

final titleStyle = GoogleFonts.notoSansSc(
  fontSize: 30,
  fontWeight: FontWeight.w800,
  letterSpacing: 2.0,
  height: 1.0,
  color: Colors.white,
);

final title = ShaderMask(
  shaderCallback: (bounds) => titleGradient.createShader(bounds),
  child: Text(titleText, style: titleStyle),
);

final strokeColor = (isDark ? Colors.black : Colors.white).withAlpha(40);
final shadowColor = (isDark ? Colors.black : Colors.white).withAlpha(22);
return RepaintBoundary(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.min,
    children: [
      Stack(
        alignment: Alignment.center,
        children: [
          Text(
            titleText,
            style: titleStyle.copyWith(
              foreground: Paint()
                ..style = PaintingStyle.stroke
                ..strokeWidth = 1.15

```

```

        ..color = strokeColor,
        shadows: [
          Shadow(
            color: shadowColor,
            blurRadius: 10,
            offset: const Offset(0, 2),
          ),
        ],
      ),
    ),
    title,
  ],
),
const SizedBox(height: 2),
Text(
  'AIR READ',
  style: GoogleFonts.montserrat(
    fontSize: 10.5,
    fontWeight: FontWeight.w600,
    letterSpacing: 2.8,
    height: 1.0,
    color: onSurface.withAlpha(isDark ? 150 : 140),
  ),
),
],
),
);
},
);
}
}

```

```

// File: lib\presentation\widgets\book_card.dart
import 'dart:async';
import 'package:flutter/material.dart';
import 'dart:math';
import 'package:provider/provider.dart';
import '../data/models/book.dart';
import '../core/theme/app_colors.dart';
import '../core/theme/app_tokens.dart';
import '../providers/read_aloud_provider.dart';
import 'local_file_image.dart'
  if (dart.library.js_interop) 'local_file_image_web.dart';
class BookCard extends StatefulWidget {

```

```

final Book book;
final VoidCallback onTap;
final int itemIndex;
final int totalColumns;
final double childAspectRatio;
// Selection Control
final bool isSelectionMode;
final bool isSelected;
final VoidCallback? onSelectionToggle;
const BookCard({
  super.key,
  required this.book,
  required this.onTap,
  this.itemIndex = 0,
  this.totalColumns = 3,
  this.childAspectRatio = 0.65,
  this.isSelectionMode = false,
  this.isSelected = false,
  this.onSelectionToggle,
});
@override
State<BookCard> createState() => _BookCardState();
}
class _BookCardState extends State<BookCard> {
  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    final scheme = theme.colorScheme;
    final isDark = theme.brightness == Brightness.dark;
    final rap = context.watch<ReadAloudProvider>();
    final bool isCurrentReadBook = rap.bookId == widget.book.id &&
      (rap.playing || rap.paused || rap.preparing);
    return GestureDetector(
      onTap: widget.onTap,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // Fixed aspect ratio for cover image
          Expanded(
            child: Container(
              decoration: BoxDecoration(
                color: scheme.surface,
                borderRadius: BorderRadius.circular(AppTokens.radiusMd),
                border: Border.all(

```

```

        color: scheme.onSurface.withOpacityCompat(isDark ? 0.16 : 0.06),
        width: AppTokens.stroke,
    ),
    boxShadow: isDark ? null : AppTokens.shadowTight,
),
child: ClipRRect(
    borderRadius: BorderRadius.circular(AppTokens.radiusMd),
    child: Stack(
        fit: StackFit.expand,
        children: [
            _buildCoverImage(),
            if (isCurrentReadBook)
                Positioned(
                    top: 8,
                    left: 8,
                    child: Material(
                        color: Colors.transparent,
                        child: InkWell(
                            borderRadius: BorderRadius.circular(999),
                            onTap: () {
                                if (widget.isSelectionMode) return;
                                if (rap.playing || rap.preparing) {
                                    unawaited(rap.pause());
                                } else if (rap.paused) {
                                    unawaited(rap.resume());
                                }
                            },
                        ),
                    child: Container(
                        constraints: const BoxConstraints(
                            minWidth: 44,
                            minHeight: 32,
                        ),
                        padding: const EdgeInsets.symmetric(
                            horizontal: 10, vertical: 7),
                        decoration: BoxDecoration(
                            color: Colors.black
                                .withOpacityCompat(isDark ? 0.55 : 0.40),
                            borderRadius: BorderRadius.circular(999),
                            border: Border.all(
                                color: Colors.white.withOpacityCompat(0.18),
                                width: 1,
                            ),
                        ),
                    ),
                child: Row(

```

```
mainAxisSize: MainAxisSize.min,
children: [
  Icon(
    Icons.volume_up_rounded,
    color: Colors.white,
    size: 14,
  ),
  const SizedBox(width: 4),
  Text(
    rap.preparing
      ? '准备中'
      : (rap.playing ? '朗读中' : '已暂停'),
    style: const TextStyle(
      color: Colors.white,
      fontSize: 11,
      fontWeight: FontWeight.w700,
      height: 1.0,
    ),
  ),
],
),
),
),
),
),
),
// Selection Overlay (WeChat Style: Bottom-Right Circle)
if (widget.isSelectionMode)
  Positioned(
    bottom: 8,
    right: 8,
    child: Container(
      width: 22,
      height: 22,
      decoration: BoxDecoration(
        color: widget.isSelected
          ? AppColors.techBlue
          : scheme.surface.withOpacityCompat(0.82),
        shape: BoxShape.circle,
        border: Border.all(
          color: widget.isSelected
            ? AppColors.techBlue
            : scheme.onSurface.withOpacityCompat(0.28),
          width: 1.5,
        ),
      ),
    ),
  ),
)
```

```

        boxShadow: [
          if (!widget.isSelected)
            BoxShadow(
              color: Colors.black.withOpacityCompat(
                isDark ? 0.24 : 0.10),
              blurRadius: 4,
              offset: const Offset(0, 2),
            )
        ],
      ),
      child: widget.isSelected
        ? const Icon(Icons.check,
            size: 14, color: Colors.white)
        : null,
    ),
  ),
],
),
),
),
),
const SizedBox(height: 12),
Text(
  widget.book.title,
  style: Theme.of(context).textTheme.bodyMedium?.copyWith(
    fontWeight: FontWeight.w600,
    fontSize: 13,
    height: 1.15,
  ),
  maxLines: 1,
  overflow: TextOverflow.ellipsis,
),
Builder(
  builder: (context) {
    final author = widget.book.author.trim();
    final isUnknown =
      author.isEmpty || author.toLowerCase() == 'unknown';
    final style = Theme.of(context).textTheme.bodySmall?.copyWith(
      color: scheme.onSurface.withOpacityCompat(0.55),
    );
    final reservedHeight = (style?.fontSize ?? 12) *
      ((style?.height ?? 1.25).clamp(1.0, 2.0)).toDouble() +
      4;
    return SizedBox(

```

```

        height: reservedHeight,
        child: isUnknown
          ? null
          : Padding(
              padding: const EdgeInsets.only(top: 4),
              child: Text(
                author,
                style: style,
                maxLines: 1,
                overflow: TextOverflow.ellipsis,
              ),
            ),
      );
    },
  ),
  const SizedBox(height: 8),
  Builder(
    builder: (context) {
      final total = widget.book.totalPages;
      final current = widget.book.currentPage;
      final double progress = total > 0 && current > 0
        ? (current / total).clamp(0.0, 1.0)
        : widget.book.percentage.clamp(0.0, 1.0);
      final barColor =
        progress > 0 ? AppColors.techBlue : Colors.transparent;
      return LinearProgressIndicator(
        value: progress,
        backgroundColor:
          scheme.onSurface.withOpacityCompat(isDark ? 0.16 : 0.08),
        valueColor: AlwaysStoppedAnimation<Color>(barColor),
        borderRadius: BorderRadius.circular(2),
        minHeight: 2,
      );
    },
  ),
  const SizedBox(height: 4),
  Align(
    alignment: Alignment.centerRight,
    child: Builder(
      builder: (context) {
        final total = widget.book.totalPages;
        final current = widget.book.currentPage;
        final bool canUsePages = total > 0 && current > 0;
        final double progress = canUsePages

```

```

        ? (current / total).clamp(0.0, 1.0)
        : widget.book.percentage.clamp(0.0, 1.0);
final bool show = progress > 0;
final String percent =
    (progress * 100).clamp(0, 100).toStringAsFixed(0);
return Visibility(
    visible: show,
    maintainState: true,
    maintainAnimation: true,
    maintainSize: true,
    child: Text(
        '$percent%',
        style: Theme.of(context).textTheme.bodySmall?.copyWith(
            color: scheme.onSurface.withOpacityCompat(0.55),
            fontSize: 10,
        ),
    ),
);
    },
),
],
),
);
}
Widget _buildCoverImage() {
    if (widget.book.coverBytes != null) {
        return Image.memory(
            widget.book.coverBytes!,
            fit: BoxFit.cover,
            width: double.infinity,
            height: double.infinity,
            errorBuilder: (context, error, stackTrace) => _buildPlaceholder(),
        );
    }
    if (widget.book.coverPath.isNotEmpty) {
        final image = buildLocalFileImage(
            path: widget.book.coverPath,
            fit: BoxFit.cover,
            width: double.infinity,
            height: double.infinity,
            errorBuilder: (context, error, stackTrace) => _buildPlaceholder(),
        );
        if (image != null) return image;
    }
}

```



```

    }
    return _buildPlaceholder();
}
Widget _buildPlaceholder() {
    final int seed = widget.book.id.hashCode ^ widget.book.title.hashCode;
    final random = Random(seed);
    final double hueA = (seed.abs() % 360).toDouble();
    final double hueB = ((hueA + 48 + (seed.abs() % 24)) % 360).toDouble();
    final Color a = HSLColor.fromAHSL(1, hueA, 0.55, 0.82).toColor();
    final Color b = HSLColor.fromAHSL(1, hueB, 0.58, 0.74).toColor();
    final Color c =
        HSLColor.fromAHSL(1, (hueA + 120) % 360, 0.40, 0.90).toColor();
    final begin = random.nextBool() ? Alignment.topLeft : Alignment.topRight;
    final end =
        random.nextBool() ? Alignment.bottomRight : Alignment.bottomLeft;
    return Container(
        decoration: BoxDecoration(
            gradient: LinearGradient(
                colors: [a, b, c],
                stops: const [0.0, 0.55, 1.0],
                begin: begin,
                end: end,
            ),
        ),
        width: double.infinity,
        height: double.infinity,
        child: Stack(
            fit: StackFit.expand,
            children: [
                Positioned(
                    left: -40,
                    top: -40,
                    child: Container(
                        width: 140,
                        height: 140,
                        decoration: BoxDecoration(
                            shape: BoxShape.circle,
                            color: Colors.white.withOpacityCompat(0.18),
                        ),
                    ),
                ),
                Positioned(
                    right: -60,
                    bottom: -60,

```

```

        child: Container(
          width: 180,
          height: 180,
          decoration: BoxDecoration(
            shape: BoxShape.circle,
            color: Colors.white.withOpacityCompat(0.12),
          ),
        ),
      ),
    Center(
      child: Icon(
        Icons.book_outlined,
        size: 48,
        color: Colors.white.withOpacityCompat(0.85),
      ),
    ),
  ],
),
);
}
}

```

```

// File: lib\presentation\widgets\glass_panel.dart
import 'dart:ui';
import 'package:flutter/material.dart';
import '../core/theme/app_tokens.dart';
/// A lightweight glassmorphism container for sheets/panels.
///
/// - Uses BackdropFilter blur + high-opacity surface tint.
/// - Avoid overdoing blur/opacity: this app's base style is "airy reading".
class GlassPanel extends StatelessWidget {
  final Widget child;
  final BorderRadius borderRadius;
  final EdgeInsetsGeometry padding;
  /// The base surface color before applying [opacity].
  final Color? surfaceColor;
  /// Surface alpha applied to [surfaceColor].
  final double opacity;
  /// Backdrop blur strength.
  final double blurSigma;
  /// Optional border. If null, a subtle top border is used.
  final Border? border;
  /// Optional shadow.
  final List<BoxShadow>? boxShadow;

```

```

const GlassPanel({
  super.key,
  required this.child,
  this.borderRadius = const BorderRadius.vertical(
    top: Radius.circular(AppTokens.radiusLg),
  ),
  this.padding = const EdgeInsets.all(0),
  this.surfaceColor,
  this.opacity = AppTokens.glassOpacity,
  this.blurSigma = AppTokens.glassBlurSigma,
  this.border,
  this.boxShadow,
});

factory GlassPanel.sheet({
  Key? key,
  required Widget child,
  EdgeInsetsGeometry padding = const EdgeInsets.all(0),
  Color? surfaceColor,
  double opacity = AppTokens.glassOpacity,
  double blurSigma = AppTokens.glassBlurSigma,
  BorderRadius borderRadius = const BorderRadius.vertical(
    top: Radius.circular(AppTokens.radiusLg),
  ),
}) {
  return GlassPanel(
    key: key,
    padding: padding,
    surfaceColor: surfaceColor,
    opacity: opacity,
    blurSigma: blurSigma,
    borderRadius: borderRadius,
    boxShadow: AppTokens.shadowSoft,
    child: child,
  );
}

@override
Widget build(BuildContext context) {
  final cs = Theme.of(context).colorScheme;
  final base = surfaceColor ?? cs.surface;
  final onSurface = cs.onSurface;
  final effectiveBorder = border ??
    Border(
      top: BorderSide(
        color: onSurface.withOpacityCompat(AppTokens.borderOpacity),

```

```

        width: AppTokens.stroke,
      ),
    );
    return ClipRRect(
      borderRadius: borderRadius,
      child: BackdropFilter(
        filter: ImageFilter.blur(sigmaX: blurSigma, sigmaY: blurSigma),
        child: Container(
          padding: padding,
          decoration: BoxDecoration(
            color: base.withOpacityCompat(opacity),
            borderRadius: borderRadius,
            border: effectiveBorder,
            boxShadow: boxShadow,
          ),
          child: child,
        ),
      ),
    );
  }
}

```

```

// File: lib\presentation\widgets\illustration_panel.dart
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:url_launcher/url_launcher.dart';
import '../core/theme/app_colors.dart';
import '../core/theme/app_tokens.dart';
import '../ai/tencentcloud/embedded_public_hunyuan_credentials.dart';
import '../providers/ai_model_provider.dart';
import '../providers/illustration_provider.dart';
import '../ai/illustration/scene_card.dart';
import 'scene_image.dart';
class IllustrationPanel extends StatefulWidget {
  final bool isDark;
  final Color bgColor;
  final Color textColor;
  final String bookId;
  final String chapterId;
  final String chapterTitle;
  final String chapterContent;
  final String? initialSelectionText;
  final bool autoGenerateFromSelection;
}

```

```

final String? onlyScenelId;
const IllustrationPanel({
  super.key,
  required this.isDark,
  required this.bgColor,
  required this.textColor,
  required this.bookId,
  required this.chapterId,
  required this.chapterTitle,
  required this.chapterContent,
  this.initialSelectionText,
  this.autoGenerateFromSelection = false,
  this.onlyScenelId,
});
@override
State<IllustrationPanel> createState() => _IllustrationPanelState();
}
class _IllustrationPanelState extends State<IllustrationPanel> {
  String _styleKey = '国风';
  String _ratioKey = '1:1';
  static const int _imageCostPoints = 20000;
  final Set<String> _pendingGenerateCardIds = {};
  String _toastText = "";
  Timer? _toastTimer;
  void _showToast(String msg) {
    final t = msg.trim();
    if (t.isEmpty) return;
    _toastTimer?.cancel();
    if (!mounted) return;
    setState(() {
      _toastText = t;
    });
    _toastTimer = Timer(const Duration(milliseconds: 2400), () {
      if (!mounted) return;
      setState(() {
        _toastText = "";
      });
    });
  }
  static const Map<String, String> _stylePrompts = {
    '国风': '古代玄幻插画，国风插画，细腻画风，柔和光影，无文字无水印',
    '水墨': '古代玄幻插画，水墨国风，留白，柔和光影，无文字无水印',
    '厚涂': '古代玄幻插画，厚涂风格，电影感光影，高细节，无文字无水印',
    '日漫': '古代玄幻插画，日系动漫风格，线条清晰，柔和光影，无文字无水印',
  }
}

```

```

        '写实': '古代玄幻插画，写实风格，电影级光影，高细节，无文字无水印',
    };

    static const Map<String, String> _ratioToResolution = {
        '1:1': '1024:1024',
        '3:4': '768:1024',
        '4:3': '1024:768',
        '9:16': '768:1280',
        '16:9': '1280:768',
    };

    double _ratioKeyToAspect(String k) {
        return switch (k) {
            '1:1' => 1.0,
            '3:4' => 3 / 4,
            '4:3' => 4 / 3,
            '9:16' => 9 / 16,
            '16:9' => 16 / 9,
            _ => 1.0,
        };
    }

    @override
    void initState() {
        super.initState();
        WidgetsBinding.instance.addPostFrameCallback(_) {
            final provider = context.read<IllustrationProvider>();
            final selection = widget.initialSelectionText?.trim() ?? "";
            if (widget.autoGenerateFromSelection && selection.isNotEmpty) {
                unawaited(provider.generateFromSelection(
                    chapterId: widget.chapterId,
                    selectionText: selection,
                ));
            }
        });
    }

    Future<void> _generateWithPoints({
        required IllustrationProvider provider,
        required AiModelProvider aiModel,
        required bool usingPersonal,
        required SceneCard card,
        required String stylePrefix,
        required String resolution,
    }) async {
        if (card.status == SceneCardStatus.generating) return;
        if (_pendingGenerateCardIds.contains(card.id)) return;
        _pendingGenerateCardIds.add(card.id);
    }

```

```

bool deducted = false;
if (!usingPersonal) {
  if (aiModel.pointsBalance < _imageCostPoints) {
    _pendingGenerateCardIds.remove(card.id);
    _showToast('积分不足，无法生成');
    return;
  }
  await aiModel.addPoints(-_imageCostPoints);
  deducted = true;
}
try {
  await provider.generateImage(
    widget.chapterId,
    card,
    stylePrefix: stylePrefix,
    resolution: resolution,
  );
  if (deducted && card.status == SceneCardStatus.failed) {
    await aiModel.addPoints(_imageCostPoints);
  }
} finally {
  _pendingGenerateCardIds.remove(card.id);
}
}
Future<bool> _confirmReanalyze() async {
  final result = await showDialog<bool>({
    context: context,
    builder: (context) {
      return AlertDialog(
        title: const Text('重新分析本章插图? '),
        content: const Text('将覆盖当前场景卡片，需要重新分析。'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(context, false),
            child: const Text('取消'),
          ),
          TextButton(
            onPressed: () => Navigator.pop(context, true),
            child: const Text('重新分析'),
          ),
        ],
      );
    },
  ));
}

```

```

        return result ?? false;
    }
Future<void> _reanalyzeChapter({
    required IllustrationProvider provider,
    required AiModelProvider aiModel,
    required bool usingPersonal,
}) async {
    if (provider.isAnalyzingOrQueued(widget.chapterId)) {
        _showToast('正在分析中...');
        return;
    }
    final content = widget.chapterContent.trim();
    if (content.isEmpty) {
        _showToast('章节内容为空');
        return;
    }
    if (!aiModel.illustrationEnabled) {
        _showToast('请先开启插图功能');
        return;
    }
    if (aiModel.source == AiModelSource.none) {
        _showToast('请先选择 AI 模型');
        return;
    }
    final ok = await _confirmReanalyze();
    if (!ok) return;
    provider.clearChapter(widget.chapterId);
    Future<String> Function(String prompt)? generateText;
    if (aiModel.source == AiModelSource.local) {
        if (!aiModel.loaded) {
            _showToast('本地模型未就绪');
            return;
        }
        generateText = (prompt) => aiModel.generate(
            prompt: prompt,
            maxTokens: 1024,
            temperature: 0.2,
        );
    }
    try {
        _showToast('开始重新分析...');
        await provider.analyzeChapter(
            chapterId: widget.chapterId,
            chapterTitle: widget.chapterTitle.trim().isEmpty

```



```

    ),
  ),
  child: Column(
    children: [
      Padding(
        padding: const EdgeInsets.fromLTRB(16, 12, 16, 8),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            if (widget.onlyScenelId == null)
              Row(
                children: [
                  if (analyzing)
                    SizedBox(
                      width: 12,
                      height: 12,
                      child: CircularProgressIndicator(
                        strokeWidth: 2,
                        color: widget.textColor
                          .withOpacityCompat(0.55),
                      ),
                    ),
                  else if (analyzingOrQueued)
                    Icon(
                      Icons.schedule_rounded,
                      size: 14,
                      color:
                        widget.textColor.withOpacity(0.5),
                    ),
                ],
              if (analyzing || analyzingOrQueued)
                Padding(
                  padding: const EdgeInsets.only(left: 6),
                  child: Text(
                    analyzing ? '分析中' : '等待分析',
                    style: TextStyle(
                      color: widget.textColor
                        .withOpacityCompat(0.6),
                      fontSize: 12,
                      height: 1.0,
                      fontWeight: FontWeight.w600,
                    ),
                  ),
                ),
            const Spacer(),

```

```

        TextButton.icon(
          onPressed: analyzingOrQueued
            ? null
            : () => unawaited(
                _reanalyzeChapter(
                  provider: provider,
                  aiModel: aiModel,
                  usingPersonal: usingPersonal,
                ),
              ),
          icon:
            const Icon(Icons.refresh_rounded, size: 16),
          label: const Text('重新分析本章'),
          style: TextButton.styleFrom(
            foregroundColor: AppColors.techBlue,
            padding: EdgeInsets.zero,
            minimumSize: const Size(0, 0),
            tapTargetSize:
              MaterialTapTargetSize.shrinkWrap,
            textStyle: const TextStyle(
              fontSize: 12,
              fontWeight: FontWeight.w700,
              height: 1.0,
            ),
          ),
        ),
      ],
    ),
    if (widget.onlyScenelId == null)
      const SizedBox(height: 8),
    _settingsRow(
      label: '风格',
      child: Wrap(
        spacing: 8,
        runSpacing: 8,
        children: _stylePrompts.keys.map((k) {
          final active = k == _styleKey;
          return _chip(
            label: k,
            active: active,
            onTap: () => setState(() => _styleKey = k),
          );
        }).toList(),
      ),
    ),
  ),

```

```

),
const SizedBox(height: 10),
_settingsRow(
  label: '画幅',
  child: Wrap(
    spacing: 8,
    runSpacing: 8,
    children: _ratioToResolution.keys.map((k) {
      final active = k == _ratioKey;
      return _chip(
        label: k,
        active: active,
        onTap: () => setState(() => _ratioKey = k),
      );
    }).toList(),
  ),
),
if (isLocal)
  Padding(
    padding: const EdgeInsets.only(top: 10),
    child: Text(
      '本地模式暂不支持插图生成功能。',
      style: TextStyle(
        color: widget.textColor.withOpacityCompat(0.65),
        fontSize: 12,
        height: 1.35,
      ),
    ),
  ),
if (!isLocal && !usingPersonal && !canGenerateImage)
  Padding(
    padding: const EdgeInsets.only(top: 10),
    child: Row(
      children: [
        Expanded(
          child: Text(
            '积分不足 2 万，无法生成插图，请先购买积分。',
            style: TextStyle(
              color: widget.textColor
                .withOpacityCompat(0.65),
              fontSize: 12,
              height: 1.35,
            ),
          ),
        ),
      ],
    ),
  ),

```

```

    ),
    TextButton(
      onPressed: () async {
        final uri = Uri.parse(
          'https://pay.ldxp.cn/item/ajnlvp');
        await launchUrl(uri,
          mode: LaunchMode.externalApplication);
      },
      style: TextButton.styleFrom(
        foregroundColor: AppColors.techBlue,
        padding: EdgeInsets.zero,
        minimumSize: const Size(0, 0),
        tapTargetSize:
          MaterialTapTargetSize.shrinkWrap,
      ),
      child: const Text(
        '购买',
        style: TextStyle(
          fontSize: 12,
          fontWeight: FontWeight.w700),
      ),
    ),
  ],
),
],
),
),
),
),
Expanded(
  child: visibleScenes.isEmpty
    ? Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          Icon(
            Icons.image_search_rounded,
            size: 48,
            color:
              widget.textColor.withOpacityCompat(0.2),
          ),
          const SizedBox(height: 12),
          Text(
            '暂无插图',
            style: TextStyle(

```

动分析场景。'

```
        color:
          widget.textColor.withOpacityCompat(0.4),
      ),
    ),
    const SizedBox(height: 16),
    Text(
      widget.onlySceneld == null
        ? '请在 AI 伴读中开启插图开关并进入章节后自
          : '未找到该插图。',
      style: TextStyle(
        color:
          widget.textColor.withOpacityCompat(0.6),
        fontSize: 12,
        height: 1.35,
      ),
      textAlign: TextAlign.center,
    ),
  ],
),
)
: ListView.separated(
  padding: const EdgeInsets.symmetric(
    horizontal: 16, vertical: 16),
  itemCount: visibleScenes.length,
  separatorBuilder: (_, __) =>
    const SizedBox(height: 16),
  itemBuilder: (context, index) {
    return _SceneCardWidget(
      card: visibleScenes[index],
      isDark: widget.isDark,
      textColor: widget.textColor,
      canGenerate: canGenerateImage,
      imageAspectRatio: selectedAspectRatio,
      onEditPrompt: () => unawaited(
        _editPromptDialog(
          provider: provider,
          card: visibleScenes[index],
        ),
      ),
    ),
  ),
  onGenerate: () => unawaited(
    _generateWithPoints(
      provider: provider,
      aiModel: aiModel,
```

```

        usingPersonal: usingPersonal,
        card: visibleScenes[index],
        stylePrefix: selectedStyle,
        resolution: selectedResolution,
      ),
    ),
  );
},
),
),
),
),
),
),
if (_toastText.isNotEmpty)
  Center(
    child: IgnorePointer(
      child: AnimatedOpacity(
        opacity: _toastText.isNotEmpty ? 1 : 0,
        duration: const Duration(milliseconds: 160),
        child: Container(
          padding: const EdgeInsets.symmetric(
            horizontal: 14, vertical: 8),
          decoration: BoxDecoration(
            color: Colors.black.withOpacityCompat(0.72),
            borderRadius: BorderRadius.circular(20),
          ),
          child: Text(
            _toastText,
            style: const TextStyle(
              color: Colors.white,
              fontSize: 13,
              height: 1.1,
              decoration: TextDecoration.none,
            ),
            textAlign: TextAlign.center,
          ),
        ),
      ),
    ),
  ),
),
);
}
);
```

```

}
Future<void> _editPromptDialog({
  required IllustrationProvider provider,
  required SceneCard card,
}) async {
  final controller = TextEditingController(text: card.action);
  try {
    await showDialog<void>(
      context: context,
      builder: (context) {
        return AlertDialog(
          title: const Text('编辑提示词'),
          content: TextField(
            controller: controller,
            minLines: 4,
            maxLines: 10,
            decoration: const InputDecoration(
              hintText: '输入用于生图的提示词',
              border: OutlineInputBorder(),
            ),
          ),
          actions: [
            TextButton(
              onPressed: () => Navigator.of(context).pop(),
              child: const Text('取消'),
            ),
            TextButton(
              onPressed: () {
                provider.updateScenePrompt(
                  chapterId: widget.chapterId,
                  sceneId: card.id,
                  prompt: controller.text,
                );
                Navigator.of(context).pop();
              },
              child: const Text('保存'),
            ),
          ],
        );
      },
    );
  } finally {
    controller.dispose();
  }
}

```



```

}
Widget _settingsRow({required String label, required Widget child}) {
  return Row(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      SizedBox(
        width: 44,
        child: Padding(
          padding: const EdgeInsets.only(top: 6),
          child: Text(
            label,
            style: TextStyle(
              fontSize: 12,
              color: widget.textColor.withOpacityCompat(0.6),
              fontWeight: FontWeight.w600,
            ),
          ),
        ),
      ),
      const SizedBox(width: 10),
      Expanded(child: child),
    ],
  );
}

Widget _chip({
  required String label,
  required bool active,
  required VoidCallback onTap,
}) {
  final bg = active ? AppColors.techBlue : widget.textColor.withOpacity(0.06);
  final fg = active ? Colors.white : widget.textColor.withOpacity(0.85);
  return InkWell(
    onTap: onTap,
    borderRadius: BorderRadius.circular(999),
    child: Container(
      padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 6),
      decoration: BoxDecoration(
        color: bg,
        borderRadius: BorderRadius.circular(999),
        border: Border.all(
          color: active
            ? AppColors.techBlue
            : widget.textColor.withOpacityCompat(0.08),
          width: AppTokens.stroke,

```

```

        ),
    ),
    child: Text(
      label,
      style: TextStyle(
        fontSize: 12,
        fontWeight: FontWeight.w700,
        color: fg,
        height: 1.0,
      ),
    ),
  ),
);
}
}

class _SceneCardWidget extends StatelessWidget {
  final SceneCard card;
  final bool isDark;
  final Color textColor;
  final VoidCallback onGenerate;
  final VoidCallback onEditPrompt;
  final double imageAspectRatio;
  final bool canGenerate;
  const _SceneCardWidget({
    required this.card,
    required this.isDark,
    required this.textColor,
    required this.onGenerate,
    required this.onEditPrompt,
    required this.imageAspectRatio,
    required this.canGenerate,
  });
  @override
  Widget build(BuildContext context) {
    // 卡片背景稍微深一点，与面板背景区分
    final bg = isDark ? Colors.white.withOpacityCompat(0.05) : Colors.white;
    return Container(
      decoration: BoxDecoration(
        color: bg,
        borderRadius: BorderRadius.circular(16),
        border: Border.all(
          color: textColor.withOpacity(0.06),
          width: AppTokens.stroke,
        ),
      ),

```

```
boxShadow: isDark
    ? []
    : [
        BoxShadow(
            color: Colors.black.withOpacity(0.02),
            blurRadius: 8,
            offset: const Offset(0, 2),
        ),
    ],
clipBehavior: Clip.antiAlias,
child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
        // 状态/图片区
        AspectRatio(
            aspectRatio: imageAspectRatio,
            child: _buildMediaArea(context),
        ),
        // 文本信息区
        Padding(
            padding: const EdgeInsets.all(14),
            child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                    Row(
                        mainAxisAlignment: MainAxisAlignment.center,
                        children: [
                            Expanded(
                                child: Text(
                                    card.title,
                                    style: TextStyle(
                                        fontSize: 15,
                                        fontWeight: FontWeight.bold,
                                        color: textColor,
                                    ),
                                ),
                            ),
                        ],
                    ),
                    if (card.status != SceneCardStatus.generating)
                        Padding(
                            padding: const EdgeInsets.only(left: 8),
                            child: IconButton(
                                onPressed: canGenerate ? onGenerate : null,
                                icon: const Icon(Icons.brush_rounded, size: 16),
```

```

        label: Text(card.status == SceneCardStatus.draft
            ? '生成'
            : '重新生成'),
        style: TextButton.styleFrom(
            visualDensity: VisualDensity.compact,
            foregroundColor: AppColors.techBlue,
            backgroundColor:
                AppColors.techBlue.withOpacityCompat(0.1),
            padding: const EdgeInsets.symmetric(
                horizontal: 12, vertical: 6),
            minimumSize: Size.zero,
            tapTargetSize: MaterialTapTargetSize.shrinkWrap,
        ),
    ),
),
],
),
const SizedBox(height: 10),
Row(
  children: [
    Text(
      '提示词',
      style: TextStyle(
        fontSize: 12,
        fontWeight: FontWeight.w700,
        color: textColor.withOpacityCompat(0.7),
      ),
    ),
    const Spacer(),
    IconButton(
      onPressed: onEditPrompt,
      icon: Icon(
        Icons.edit_outlined,
        size: 18,
        color: textColor.withOpacityCompat(0.65),
      ),
      visualDensity: VisualDensity.compact,
      padding: EdgeInsets.zero,
      constraints: const BoxConstraints(),
      tooltip: '编辑提示词',
    ),
  ],
),
const SizedBox(height: 6),

```

```

GestureDetector(
  onTap: onEditPrompt,
  child: Text(
    card.action.trim().isEmpty ? '（暂无）': card.action,
    style: TextStyle(
      fontSize: 12,
      height: 1.35,
      color: textColor.withOpacityCompat(0.75),
    ),
    maxLines: 6,
    overflow: TextOverflow.ellipsis,
  ),
),
],
),
],
),
),
);
}
Widget _buildMediaArea(BuildContext context) {
  switch (card.status) {
    case SceneCardStatus.draft:
      return Container(
        color: textColor.withOpacityCompat(0.03),
        child: Stack(
          alignment: Alignment.center,
          children: [
            Icon(
              Icons.image_outlined,
              size: 48,
              color: textColor.withOpacityCompat(0.1),
            ),
            Positioned(
              bottom: 16,
              child: Text(
                '点击生成预览画面',
                style: TextStyle(
                  fontSize: 12,
                  color: textColor.withOpacityCompat(0.4),
                ),
              ),
            ),
          ],
        ),
      );
  }
}

```

```

    ),
  );
case SceneCardStatus.generating:
  return Container(
    color: textColor.withOpacityCompat(0.03),
    child: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          SizedBox(
            width: 24,
            height: 24,
            child: CircularProgressIndicator(
              strokeWidth: 2,
              color: AppColors.techBlue,
            ),
          ),
          const SizedBox(height: 12),
          Text(
            'AI 正在绘图...',
            style: TextStyle(
              fontSize: 12,
              color: textColor.withOpacityCompat(0.6),
            ),
          ),
        ],
      ),
    ),
  );
case SceneCardStatus.completed:
  if (card.localImagePath != null) {
    return GestureDetector(
      onTap: () =>
        _openImagePreview(context: context, path: card.localImagePath!),
      child: buildSceneImage(
        card.localImagePath!,
        fit: BoxFit.contain,
      ),
    );
  }
  return const Center(child: Text('图片文件丢失'));
case SceneCardStatus.failed:
  return Container(
    color: Colors.red.withOpacityCompat(0.05),

```

```

        child: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
              const Icon(Icons.error_outline, color: Colors.red, size: 32),
              const SizedBox(height: 8),
              Text(
                '生成失败',
                style: TextStyle(color: Colors.red.withOpacityCompat(0.8)),
              ),
              if (card.errorMsg != null)
                Padding(
                  padding: const EdgeInsets.symmetric(horizontal: 20),
                  child: Text(
                    card.errorMsg!,
                    style: const TextStyle(fontSize: 10, color: Colors.red),
                    maxLines: 1,
                    overflow: TextOverflow.ellipsis,
                  ),
                ),
              const SizedBox(height: 8),
              TextButton(
                onPressed: canGenerate ? onGenerate : null,
                child: const Text('重新生成'),
              ),
            ],
          ),
        ),
      );
    }
  }

  Future<void> _openImagePreview({
    required BuildContext context,
    required String path,
  }) async {
    await showDialog<void>(
      context: context,
      barrierColor: Colors.black.withOpacityCompat(0.92),
      builder: (context) {
        return Dialog.fullscreen(
          backgroundColor: Colors.black,
          child: SafeArea(
            child: Stack(
              children: [

```

```

        Positioned.fill(
          child: InteractiveViewer(
            minScale: 1,
            maxScale: 4,
            child: Center(
              child: buildSceneImage(
                path,
                fit: BoxFit.contain,
              ),
            ),
          ),
        ),
      ),
    Positioned(
      top: 8,
      right: 8,
      child: IconButton(
        onPressed: () => Navigator.of(context).pop(),
        icon: const Icon(Icons.close, color: Colors.white),
      ),
    ),
  ],
),
);
},
);
}
}
class _Tag extends StatelessWidget {
  final String text;
  final Color color;
  const _Tag({required this.text, required this.color});
  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
      decoration: BoxDecoration(
        color: color.withOpacityCompat(0.05),
        borderRadius: BorderRadius.circular(6),
        border: Border.all(
          color: color.withOpacityCompat(0.05),
          width: 0.5,
        ),
      ),
    ),
  ),

```



```

        child: Text(
          text,
          style: TextStyle(
            fontSize: 11,
            color: color.withOpacityCompat(0.6),
          ),
        ),
      );
    }
  }
}

```

// File: lib\presentation\widgets\local_file_image.dart

```

import 'dart:io';
import 'package:flutter/material.dart';
Widget? buildLocalFileImage({
  required String path,
  required BoxFit fit,
  double? width,
  double? height,
  ImageErrorWidgetBuilder? errorBuilder,
}) {
  return Image.file(
    File(path),
    fit: fit,
    width: width,
    height: height,
    errorBuilder: errorBuilder,
  );
}

```

// File: lib\presentation\widgets\local_file_image_web.dart

```

import 'package:flutter/material.dart';
Widget? buildLocalFileImage({
  required String path,
  required BoxFit fit,
  double? width,
  double? height,
  ImageErrorWidgetBuilder? errorBuilder,
}) {
  return null;
}

```

// File: lib\presentation\widgets\mobile_import_overlay.dart

```

import 'package:flutter/material.dart';

```

```

import 'package:flutter_animate/flutter_animate.dart';
import '../core/theme/app_colors.dart';
import '../core/theme/app_tokens.dart';
class MobileImportOverlay extends StatelessWidget {
  const MobileImportOverlay({super.key});
  @override
  Widget build(BuildContext context) {
    return Container(
      color: Colors.black.withOpacityCompat(0.7),
      alignment: Alignment.center,
      child: Container(
        padding: const EdgeInsets.symmetric(horizontal: 32, vertical: 24),
        decoration: BoxDecoration(
          color: Colors.transparent,
          borderRadius: BorderRadius.circular(16),
        ),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            // Circular Progress with Gradient
            Container(
              width: 60,
              height: 60,
              decoration: BoxDecoration(
                shape: BoxShape.circle,
                gradient: const LinearGradient(
                  colors: [AppColors.techBlue, AppColors.airBlue],
                  begin: Alignment.topLeft,
                  end: Alignment.bottomRight,
                ),
              ),
              boxShadow: [
                BoxShadow(
                  color: AppColors.techBlue.withOpacityCompat(0.4),
                  blurRadius: 10,
                  spreadRadius: 2,
                ),
              ],
            ),
            child: const Padding(
              padding: EdgeInsets.all(4.0), // Border width
              child: CircularProgressIndicator(
                color: Colors.white,
                strokeWidth: 3,
              ),
            ),
          ],
        ),
      ),
    );
  }
}

```

```

        ),
      ),
      .animate(onPlay: (controller) => controller.repeat(reverse: true))
      .fade(begin: 0.6, end: 1.0, duration: 1000.ms), // Breathing effect
      const SizedBox(height: 24),
      // Text
      const Text(
        '正在后台导入中，完成后将自动刷新',
        style: TextStyle(
          color: Colors.white,
          fontSize: 16,
          fontWeight: FontWeight.w500,
          letterSpacing: 0.5,
        ),
        textAlign: TextAlign.center,
      ),
    ],
  ),
).animate().scale(
  begin: const Offset(0.8, 0.8),
  end: const Offset(1.0, 1.0),
  duration: 300.ms,
  curve: Curves.easeOutBack,
),
);
}
}

```

```

// File: lib\presentation\widgets\scene_image.dart
export 'scene_image_io.dart'
  if (dart.library.js_interop) 'scene_image_web.dart';

```

```

// File: lib\presentation\widgets\scene_image_io.dart
import 'dart:io';
import 'package:flutter/material.dart';
Widget buildSceneImage(String path, {BoxFit fit = BoxFit.cover}) {
  final p = path.trim();
  if (p.startsWith('http://') || p.startsWith('https://')) {
    return Image.network(
      p,
      fit: fit,
      errorBuilder: (_, __, ___) =>
        const Center(child: Icon(Icons.broken_image)),
    );
  }
}

```

```

    }
    return Image.file(
      File(p),
      fit: fit,
      errorBuilder: (_, __, ___) => const Center(child: Icon(Icons.broken_image)),
    );
  }
}

```

```

// File: lib\presentation\widgets\scene_image_web.dart
import 'package:flutter/material.dart';

Widget buildSceneImage(String path, {BoxFit fit = BoxFit.cover}) {
  final p = path.trim();
  if (p.startsWith('http://') || p.startsWith('https://')) {
    return Image.network(
      p,
      fit: fit,
      errorBuilder: (_, __, ___) =>
        const Center(child: Icon(Icons.broken_image)),
    );
  }
  return Container(
    color: Colors.black.withOpacity(0.04),
    alignment: Alignment.center,
    child: const Icon(Icons.image_not_supported_outlined),
  );
}

```